

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
ĐẠI HỌC BÁCH KHOA THÀNH PHỐ HỒ CHÍ MINH
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ án chuyên ngành

Báo cáo đồ án chuyên ngành

Trục tích hợp dữ liệu (Data Integration Hub)

Giảng viên hướng dẫn: ThS. Nguyễn Cao Đạt

ThS. Nguyễn Quang Sang

Sinh viên thực hiện: Trần Mạnh Tuấn - 2213807

Phạm Lê Huy - 2252260

HỒ CHÍ MINH, THÁNG 5 2026



Mục lục

Danh mục thuật ngữ và từ viết tắt	8
1 Tổng quan	10
2 Phân tích nghiệp vụ và đặc trưng dữ liệu của trường đại học	12
3 Những vấn đề trong tích hợp dữ liệu và cách giải quyết	15
3.1 Các dạng xung đột dữ liệu thường gặp	15
3.2 Cách giải quyết	17
3.2.1 Tầng tiếp nhận dữ liệu — <code>DataIntegrationService</code>	18
3.2.2 Tầng đồng bộ — <code>SyncEngineService</code>	19
3.2.3 Tổng hợp cơ chế phòng thủ	21
4 Thiết kế kiến trúc	21
4.1 Yêu Cầu Chức Năng và Phi Chức Năng	21
4.2 Sơ Đồ Kiến Trúc Tổng Thể	23
4.3 Giải Thích Chi Tiết Các Thành Phần	24
4.3.1 Admin Dashboard (Next.js Frontend)	24
4.3.2 Kong API Gateway	24
4.3.3 NestJS Backend (Monolith)	24
4.3.4 Tầng Lưu Trữ	26
4.3.5 Hệ Thống Nguồn Dữ Liệu	26
4.3.6 Giám Sát: Prometheus và Grafana	26
4.4 So Sánh Với Kiến Trúc Lý Thuyết Ban Đầu	27
5 Cơ chế xác thực và phân quyền	28
5.1 Luồng xác thực	28
5.2 Luồng phân quyền	29
5.2.1 Lớp 1 — Kiểm tra vai trò (<code>RolesGuard</code>)	29
5.2.2 Lớp 2 — Kiểm tra phạm vi bảng (<code>PermissionsGuard</code>)	30
5.3 Cơ chế cập nhật mã truy cập	31
5.4 Cấu trúc cơ sở dữ liệu phục vụ xác thực và phân quyền	31
6 Luồng xử lý dữ liệu	33
6.1 Chiến lược đồng bộ dữ liệu và cơ chế ghi vào cơ sở dữ liệu	33
6.1.1 Tổng quan	33

6.1.2	Quy trình chung: Phân trang và tải dữ liệu	33
6.1.3	Chiến lược <code>upsert</code> — Mặc định	35
6.1.4	Chiến lược <code>incremental</code> — Bộ lọc tại phía máy chủ	37
6.1.5	Chiến lược <code>overwrite</code> — Xóa và ghi đè toàn bộ	38
6.2	Cơ Chế Retry Trong ETL Pipeline	40
6.2.1	Đặt Vấn Đề	40
6.2.2	Thiết Kế Retry Có Chủ Ý	40
6.2.3	Luồng Xử Lý Khi Retry	42
6.2.4	Đánh Giá	42
6.3	Phát hiện bản ghi mồ côi (Orphan Detection)	44
6.3.1	Đặt vấn đề	44
6.3.2	Giải pháp: Phép trừ tập hợp khóa chính	44
6.3.3	Quy trình triển khai	44
6.3.4	Thiết kế không tự động xóa	45
6.3.5	API quản lý bản ghi mồ côi	46
6.3.6	Giới hạn của phương pháp	47
6.4	Thông Báo Qua Email Sau Khi Pipeline Chạy Xong	48
6.4.1	Đặt vấn đề	48
6.4.2	Công nghệ sử dụng	48
6.4.3	Điểm kích hoạt	48
6.4.4	Nội dung email	48
6.4.5	Cấu hình biến môi trường	49
6.4.6	Luồng xử lý	50
7	Chiến Lược Sao Lưu Dữ Liệu Với MinIO và AWS S3	51
7.1	Đặt Vấn Đề	51
7.2	Kiến Trúc Hai Tầng: MinIO và AWS S3	51
7.2.1	Cấu Hình Triển Khai	52
7.3	Bốn Loại Trigger Backup	53
7.3.1	Trigger 1: Pre-Sync Backup	53
7.3.2	Trigger 2: Schema Change Backup	53
7.3.3	Trigger 3: Scheduled Weekly Backup	53
7.3.4	Trigger 4: Manual Backup	54
7.4	Cấu Trúc Lưu Trữ Trong MinIO	54
7.5	Chính Sách Lưu Giữ (Retention Policy)	56
7.6	Cơ Chế Đồng Bộ Lên AWS S3	57



7.6.1	Mục Tiêu	57
7.6.2	Luồng Hoạt Động	57
7.6.3	Trạng Thái Đồng Bộ	58
7.6.4	Fire-and-Forget cho Sync Hàng Loạt	58
7.7	API Quản Lý Backup	59
7.8	Cơ Chế Restore	59
7.9	Đánh Giá	60
7.9.1	Ưu Điểm	60
7.9.2	Hạn Chế và Hướng Phát Triển	60
8	Phân tích Hiệu năng và Capacity Planning	61
8.1	Tổng quan	61
8.2	Kiểm thử Data Collision — Mock Source Server	61
8.2.1	Kiến trúc Mock Server	61
8.2.2	Ma trận Test Case	62
8.2.3	Ví dụ kết quả kiểm thử — TC-03 Deduplication	64
8.2.4	Ví dụ kết quả kiểm thử — TC-11 Dead Letter	64
8.3	Kiểm thử Hiệu năng — PERF Test Cases	65
8.3.1	Thiết kế MemoryProfiler	65
8.3.2	PERF-01 — RAM Capacity Planning	66
8.3.3	PERF-02 — Incremental Sync với Server-side Filter	67
8.4	Phân tích RAM cho Sync Job	69
8.4.1	Vòng đời dữ liệu trong RAM	69
8.4.2	Ước lượng RAM theo kích thước bảng	69
8.5	Phân tích Throughput cho Export API (Outbound)	69
8.5.1	Kiến trúc Export API hiện tại	69
8.5.2	Bottleneck: OFFSET Pagination	69
8.5.3	Ước lượng Throughput theo Little's Law	70
8.6	Tóm tắt và Khuyến nghị Server	71
9	Giao diện người dùng và quản trị hệ thống	71
9.1	Bảng điều khiển trung tâm	72
9.2	Quản lý lịch sử đồng bộ	72
9.3	Quản lý sao lưu và phục hồi	73
9.4	Quản lý phân quyền truy cập	75
9.5	Quản lý và đối soát lược đồ dữ liệu	76

Danh sách hình vẽ

4.1	Luồng cấp lại mã truy cập	23
5.1	Luồng xác thực và kiểm tra quyền truy cập	29
5.2	Luồng xử lý cấp lại mã truy cập (Refresh Token)	31
6.1	Cách thức hoạt động của hàm syncOneTable	36
6.2	Luồng quyết định retry trong pipeline ETL	42
6.3	Quy trình hai bước: phát hiện và xác nhận xóa bản ghi mồ côi	46
6.4	Minh họa mail cảnh báo	49
6.5	Luồng gửi email thông báo sau khi pipeline kết thúc	50
7.1	Kiến trúc lưu trữ hai tầng: MinIO và AWS S3	52
7.2	Giao diện cấu trúc trong MINIO	55
7.3	Các file Backup cho một bảng trong MINIO	55
7.4	Giao diện lưu trữ file trên S3	57
9.1	Giao diện bảng điều khiển trung tâm	73
9.2	Giao diện quản lý lịch sử đồng bộ	74
9.3	Giao diện quản lý sao lưu và phục hồi dữ liệu	75
9.4	Giao diện quản lý phân quyền truy cập cho người dùng	76
9.5	Giao diện quản lý lược đồ dữ liệu	77

Danh sách bảng

3.1	Các trường hợp xung đột kiểu dữ liệu điển hình	16
3.2	Quy tắc chuẩn hóa kiểu dữ liệu	20
3.3	Tổng hợp các cơ chế phòng thủ xung đột dữ liệu	21
4.1	So sánh kiến trúc lý thuyết và thực tế	27
6.1	Hiệu quả Incremental: Lọc tại đích (BE-side) và lọc tại nguồn (Server-side)	38
6.2	Chính sách retry theo loại lỗi	41
6.3	Biến môi trường cho NotificationService	49
7.1	Chính sách lưu giữ backup theo loại trigger	56
7.2	Các giá trị trạng thái đồng bộ S3	58
7.3	Các endpoint quản lý backup	59
8.1	Ma trận kiểm thử Data Collision Defense	63
8.2	Phân tích nguồn tiêu thụ RAM trong pipeline (PERF-01)	67
8.3	So sánh upsert vs incremental — kết quả thực nghiệm	68



8.4	Ước lượng peak RAM theo cấu hình bảng	69
8.5	Ước lượng latency theo độ sâu trang — OFFSET vs Keyset	70
8.6	Ước lượng RPS tối đa theo cấu hình pool và latency	71
8.7	Khuyến nghị cấu hình server cho Data Integration Hub	71

Danh mục thuật ngữ và từ viết tắt

Từ viết tắt/Thuật ngữ	Ý nghĩa
ACID	Atomicity, Consistency, Isolation, Durability - Các thuộc tính đảm bảo độ tin cậy của giao dịch cơ sở dữ liệu.
API	Application Programming Interface - Giao diện lập trình ứng dụng.
CI/CD	Continuous Integration/Continuous Deployment - Tích hợp liên tục và Triển khai liên tục.
Data Silo	Ốc đảo dữ liệu - Tình trạng dữ liệu bị cô lập, phân mảnh giữa các hệ thống.
ETL	Extract, Transform, Load - Quá trình trích xuất, biến đổi và tải dữ liệu.
Golden Record	Bản ghi lõi/Bản ghi vàng - Phiên bản dữ liệu duy nhất, đầy đủ và đáng tin cậy nhất sau khi đã qua chuẩn hóa từ nhiều nguồn.
HTTP	Hypertext Transfer Protocol - Giao thức truyền tải siêu văn bản.
JSON	JavaScript Object Notation - Định dạng trao đổi dữ liệu.
JWT	JSON Web Token - Định dạng token để xác thực và trao đổi thông tin.
MDM	Master Data Management - Quản trị dữ liệu chủ.
OIDC	OpenID Connect - Lớp xác thực người dùng trên nền tảng OAuth 2.0.
ORM	Object-Relational Mapping - Kỹ thuật ánh xạ dữ liệu giữa cơ sở dữ liệu quan hệ và đối tượng trong code.



RBAC	Role-Based Access Control - Phân quyền truy cập dựa trên vai trò.
Schema Registry	Nơi lưu trữ, quản lý cấu trúc siêu dữ liệu (metadata) của các bảng.
SSO	Single Sign-On - Cơ chế đăng nhập một lần.
UI	User Interface - Giao diện người dùng.
Upsert	Update or Insert - Thao tác cập nhật bản ghi nếu đã tồn tại, hoặc tạo mới nếu chưa tồn tại.
Prisma DMMF	Data Model Meta Format — một cấu trúc dữ liệu (JSON) nội bộ mà Prisma ORM tạo ra từ file schema.prisma

1 Tổng quan

Trong bối cảnh chuyển đổi số đang diễn ra mạnh mẽ tại các cơ quan quản lý và các cơ sở giáo dục đại học, dữ liệu ngày càng trở thành tài sản cốt lõi của nhà trường. Việc quản lý, tích hợp và xây dựng các quy chuẩn dùng chung cho dữ liệu có vai trò đặc biệt quan trọng, nhằm bảo đảm hoạt động phối hợp thông suốt giữa các đơn vị, đồng thời tạo nền tảng cho công tác thống kê, phân tích và hỗ trợ ban lãnh đạo ra quyết định.

Tuy nhiên, quá trình khai thác và chia sẻ dữ liệu trong nội bộ trường đại học hiện nay vẫn còn nhiều hạn chế. Mỗi đơn vị, phòng ban thường sử dụng một hệ thống phần mềm riêng để quản lý các lĩnh vực như nhân sự, đào tạo, tài chính, nghiên cứu khoa học hoặc sinh viên. Điều này dẫn đến hiện tượng dữ liệu bị cô lập giữa các hệ thống, mỗi nơi chỉ lưu giữ một phần thông tin và thiếu sự liên kết với các hệ thống khác. Bên cạnh đó, cùng một đối tượng nhưng có thể được lưu trữ nhiều lần ở các hệ thống khác nhau với định dạng, cấu trúc hoặc nội dung không thống nhất. Khi dữ liệu được chia sẻ thủ công giữa các đơn vị, việc cập nhật thường không được đồng bộ, dễ phát sinh sai lệch, trùng lặp và khó xác định đâu là phiên bản mới nhất.

Để giải quyết bài toán trên, đề tài hướng tới việc xây dựng một trục tích hợp dữ liệu cho trường đại học. Hệ thống đóng vai trò là đầu mối trung tâm, tiếp nhận dữ liệu từ các hệ thống nguồn, thực hiện chuẩn hóa, kiểm tra tính hợp lệ và lưu trữ vào cơ sở dữ liệu dùng chung của toàn trường. Các hệ thống thành phần sẽ không kết nối trực tiếp với nhau mà trao đổi thông tin thông qua trục tích hợp, từ đó giúp giảm sự phụ thuộc lẫn nhau và bảo đảm tính nhất quán của dữ liệu.

Kiến trúc được đề xuất theo mô hình Hub-and-Spoke kết hợp với API-led, trong đó trục tích hợp là trung tâm kết nối, còn các hệ thống quản lý của từng đơn vị là các vệ tinh. Một điểm quan trọng của hệ thống là xây dựng bộ đặc tả dữ liệu thống nhất, quy định rõ cấu trúc, ý nghĩa và kiểu dữ liệu của từng trường thông tin mà các hệ thống nguồn phải tuân thủ. Trên cơ sở đó, hệ thống cung cấp các giao diện lập trình ứng dụng thống nhất để các đơn vị có thể cập nhật và khai thác dữ liệu theo cùng một chuẩn.

Ngoài chức năng tích hợp dữ liệu, trục tích hợp còn hỗ trợ công tác quản trị dữ liệu thông qua các cơ chế theo dõi, ghi nhận lịch sử thay đổi, kiểm tra trạng thái đồng bộ và giám sát hoạt động của toàn hệ thống theo thời gian thực. Đồng thời, hệ thống cũng được thiết kế với các cơ chế bảo mật như xác thực, phân quyền, mã hóa dữ liệu và cổng quản lý truy cập nhằm bảo đảm an toàn cho dữ liệu khi trao đổi giữa các hệ thống.

Bên cạnh việc phục vụ nhu cầu chia sẻ dữ liệu trong nội bộ trường, trục tích hợp còn là nền tảng để đồng bộ dữ liệu với các đơn vị quản lý cấp trên như Đại học Quốc gia Thành phố Hồ Chí Minh hoặc Hệ thống cơ sở dữ liệu giáo dục đại học. Qua đó, nhà



trường có thể bảo đảm dữ liệu được cung cấp đầy đủ, chính xác và kịp thời theo đúng yêu cầu quản lý.

2 Phân tích nghiệp vụ và đặc trưng dữ liệu của trường đại học

1. Tính chu kỳ và sự biến động theo thời gian

Dữ liệu trong trường đại học không phát sinh đồng đều mà phụ thuộc chặt chẽ vào lịch trình của năm học. Ở một số thời điểm, hệ thống có thể phải xử lý khối lượng dữ liệu rất lớn trong thời gian ngắn.

Các giai đoạn có lưu lượng truy cập cao bao gồm:

- Đăng ký học phần: Đây là thời điểm hệ thống chịu tải lớn nhất do số lượng truy vấn và cập nhật tăng đột biến trong thời gian ngắn.
- Kỳ thi và nhập điểm: Cuối mỗi học kỳ, lượng dữ liệu điểm số từ các hệ thống đào tạo được chuyển về trực tích hợp rất lớn.
- Tuyển sinh: Trong giai đoạn từ tháng 7 đến tháng 9, hệ thống phải tiếp nhận số lượng lớn hồ sơ sinh viên mới.

Ngược lại, trong thời gian nghỉ hè hoặc giữa học kỳ, dữ liệu phát sinh rất ít và tần suất cập nhật thấp. Vì vậy, trực tích hợp cần được thiết kế để có khả năng mở rộng linh hoạt theo từng thời điểm, tránh quá tải vào giai đoạn cao điểm nhưng vẫn tiết kiệm tài nguyên khi nhu cầu thấp.

2. Sự thay đổi vai trò của đối tượng

Một đặc trưng quan trọng của dữ liệu đại học là cùng một cá nhân có thể thay đổi vai trò trong suốt quá trình gắn bó với nhà trường nhưng vẫn phải được quản lý như một đối tượng duy nhất.

Ví dụ, một người có thể trải qua các giai đoạn:

Sinh viên đại học → Cựu sinh viên → Học viên cao học → Giảng viên hoặc nhân viên chức.

Mỗi giai đoạn có thể được quản lý bởi một hệ thống khác nhau và sử dụng các mã định danh khác nhau, chẳng hạn mã số sinh viên đại học, mã học viên cao học hoặc mã viên chức. Nếu không có cơ chế liên kết, dữ liệu của cùng một người sẽ bị tách rời thành nhiều bản ghi khác nhau.

Do đó, trực tích hợp cần xây dựng một định danh duy nhất cho mỗi cá nhân, có thể dựa trên căn cước công dân hoặc mã định danh nội bộ. Định danh này được sử

dụng để liên kết toàn bộ lịch sử và vai trò của đối tượng trong các hệ thống khác nhau.

3. Tính đa dạng về cấu trúc dữ liệu

Dữ liệu của trường đại học rất đa dạng về nội dung và định dạng. Một hệ thống tích hợp phải có khả năng xử lý đồng thời nhiều loại dữ liệu khác nhau.

Các nhóm dữ liệu tiêu biểu bao gồm:

- Dữ liệu định danh: họ tên, ngày sinh, giới tính, quê quán, địa chỉ.
- Dữ liệu học tập: học phần, số tín chỉ, điểm số, kết quả học tập.
- Dữ liệu tài chính: học phí, học bổng, các khoản thu chi.
- Dữ liệu nghiên cứu khoa học: bài báo, đề tài, minh chứng, tệp tài liệu.

Trong đó, có những dữ liệu được tổ chức theo cấu trúc chặt chẽ dưới dạng bảng, nhưng cũng có những dữ liệu tồn tại dưới dạng tệp văn bản, hình ảnh hoặc tài liệu đính kèm. Vì vậy, trục tích hợp cần hỗ trợ cả dữ liệu có cấu trúc và dữ liệu phi cấu trúc.

4. Tính phụ thuộc và yêu cầu nhất quán giữa các hệ thống

Dữ liệu của một đơn vị thường là đầu vào cho hoạt động của đơn vị khác. Do đó, sự thay đổi dữ liệu ở hệ thống nguồn phải được phản ánh kịp thời đến các hệ thống liên quan.

Ví dụ, sinh viên chỉ được phép đóng học phí nếu đã hoàn tất đăng ký học phần. Khi thông tin đăng ký học phần thay đổi, dữ liệu tương ứng trong hệ thống tài chính, thư viện hoặc ứng dụng dành cho sinh viên cũng cần được cập nhật.

Vì vậy, trục tích hợp cần hỗ trợ cơ chế xử lý theo sự kiện. Khi có một thay đổi xảy ra ở hệ thống nguồn, trục tích hợp sẽ tự động phát sinh các tác vụ đồng bộ đến các hệ thống phụ thuộc nhằm bảo đảm dữ liệu luôn nhất quán.

5. Tính phân cấp và nhu cầu tổng hợp dữ liệu

Dữ liệu trong trường đại học thường được hình thành theo nhiều cấp độ, từ dữ liệu chi tiết đến dữ liệu tổng hợp để phục vụ công tác thống kê và báo cáo.

Ví dụ:

Điểm thành phần → Điểm học phần → Điểm trung bình tích lũy → Xếp loại học lực.

Trực tích hợp không chỉ lưu trữ dữ liệu chi tiết mà còn cần có khả năng tính toán, tổng hợp và chuẩn hóa dữ liệu ở mức cao hơn. Điều này đặc biệt quan trọng khi đồng bộ dữ liệu lên các hệ thống cấp trên, vì các hệ thống này thường chỉ yêu cầu dữ liệu tổng hợp thay vì toàn bộ dữ liệu chi tiết.

6. Yêu cầu khác nhau về độ trễ đồng bộ

Không phải mọi loại dữ liệu đều cần được đồng bộ ngay lập tức. Tùy theo mức độ quan trọng của nghiệp vụ, mỗi loại dữ liệu sẽ có yêu cầu khác nhau về thời gian cập nhật.

- Dữ liệu điểm thi hoặc báo cáo thống kê có thể chấp nhận độ trễ từ vài chục phút đến cuối ngày.
- Dữ liệu về tình trạng sinh viên như thôi học, bảo lưu hoặc bị đình chỉ cần được cập nhật gần như ngay lập tức để kịp thời khóa hoặc thay đổi quyền truy cập của sinh viên trên các hệ thống khác.

Vì vậy, trực tích hợp cần phân loại dữ liệu theo mức độ ưu tiên để lựa chọn cơ chế đồng bộ phù hợp, tránh lãng phí tài nguyên nhưng vẫn đáp ứng yêu cầu nghiệp vụ.

3 Những vấn đề trong tích hợp dữ liệu và cách giải quyết

3.1 Các dạng xung đột dữ liệu thường gặp

Trong quá trình xây dựng trực tích hợp dữ liệu, hệ thống phải đối mặt với nhiều dạng xung đột phát sinh tại ranh giới giữa các hệ thống nguồn phân tán và cơ sở dữ liệu chủ (Master Data). Phần này trình bày các dạng xung đột đã được nhận diện và cơ chế phòng thủ tương ứng đã được hiện thực trong dự án.

1. Xung đột cấu trúc phản hồi API

Dạng xung đột này xảy ra khi hệ thống nguồn trả về dữ liệu không tuân thủ định dạng chuẩn mà trực tích hợp kỳ vọng. Đây là dạng xung đột ở tầng tiếp nhận dữ liệu — nguy hiểm nhất vì nếu không được phát hiện sớm, dữ liệu sai định dạng sẽ lan truyền sâu vào luồng xử lý, gây ra các lỗi rất khó truy vết.

Trong dự án, cấu trúc phản hồi (response) chuẩn được quy định như sau:

```
{
  "success": true,
  "metadata": { "totalPages": N, "currentPage": P, ... },
  "payload": [ { ... }, { ... } ]
}
```

Trong đó:

- **success**: Cho biết yêu cầu có được xử lý thành công hay không.
- **timestamp**: Thời điểm hệ thống trả về dữ liệu.
- **metadata**: Chứa thông tin mô tả như tên bảng dữ liệu, số lượng bản ghi, số trang và trang hiện tại.
- **payload**: Chứa dữ liệu thực tế được trả về.

Các vi phạm hợp đồng giao tiếp phổ biến được ghi nhận bao gồm:

- Thiếu trường **success** hoặc **success = false** — hệ thống nguồn đang trong trạng thái lỗi nội bộ.

- Trường `payload` không tồn tại hoặc không phải kiểu mảng (`Array`) — lỗi tuần tự hóa (serialization) ở hệ thống nguồn.
- Sử dụng tên trường khác (`data`, `records`) thay vì `payload` — sự thiếu thống nhất giữa các hệ thống nguồn.

2. Xung đột lược đồ dữ liệu

Xung đột lược đồ (schema) xảy ra khi hệ thống nguồn thay đổi cấu trúc bảng (thêm, sửa, xóa cột) mà không thông báo cho trực tích hợp. Hậu quả trực tiếp là các trường không xác định được đưa vào Prisma ORM gây lỗi *“Unknown field”*, hoặc ngược lại, các trường mới ở nguồn không được ánh xạ vào cơ sở dữ liệu chủ dẫn đến việc mất mát dữ liệu một cách thầm lặng.

3. Xung đột kiểu dữ liệu

Xung đột kiểu dữ liệu phát sinh do sự không đồng nhất trong cách các hệ thống nguồn biểu diễn cùng một giá trị. Các trường hợp thực tế ghi nhận trong dự án bao gồm:

Bảng 3.1: Các trường hợp xung đột kiểu dữ liệu điển hình

Trường	Kiểu Prisma	Nguồn gửi	Hậu quả nếu không xử lý
<code>id</code>	<code>Int</code>	"701" (chuỗi)	Lỗi kiểu dữ liệu Prisma
<code>active</code>	<code>Boolean</code>	"true" (chuỗi)	Lỗi kiểu dữ liệu Prisma
<code>ngaySinh</code>	<code>DateTime</code>	"1990-06-15T..." (chuỗi)	Lỗi kiểu dữ liệu Prisma
<code>ngaySinh</code>	<code>DateTime</code>	"not-a-date" (chuỗi)	Crash hệ thống (Runtime)

4. Xung đột khóa chính

Xung đột khóa chính xảy ra dưới hai hình thức:

- Trùng lặp trong cùng một lô dữ liệu (batch):** Hệ thống nguồn trả về nhiều bản ghi có cùng giá trị khóa chính trong một lần phân trang. Nếu lần lượt thực hiện lệnh `upsert` theo thứ tự xuất hiện, các bản ghi đầu sẽ bị ghi đè bởi bản ghi sau mà không có cảnh báo.
- Thiếu hoặc rỗng khóa chính:** Bản ghi không mang giá trị khóa chính hợp lệ, không thể thực hiện `upsert` và nếu không được xử lý sẽ bị bỏ qua một cách thầm lặng.

5. Xung đột phân trang

Đây là dạng xung đột đặc thù của kiến trúc tích hợp theo lô. Khi luồng xử lý gửi yêu cầu lấy trang thứ p , hệ thống nguồn có thể trả về dữ liệu của một trang khác do lỗi bộ nhớ đệm (cache CDN) hoặc lỗi định tuyến của bộ cân bằng tải. Hậu quả là dữ liệu ở các trang bị bỏ qua, trong khi luồng xử lý không nhận ra sự sai lệch và vẫn báo cáo thành công.

6. Xung đột ràng buộc cơ sở dữ liệu

Xung đột ràng buộc phát sinh khi dữ liệu từ nguồn vi phạm các quy tắc toàn vẹn được định nghĩa trong cơ sở dữ liệu chủ, ví dụ: giá trị vượt độ dài `VARCHAR`, vi phạm ràng buộc `NOT NULL`, hay vi phạm ràng buộc khóa ngoại. Nếu một bản ghi bị lỗi gây hủy (abort) toàn bộ lô dữ liệu, những bản ghi hợp lệ còn lại trong cùng lô đó sẽ không được đồng bộ.

3.2 Cách giải quyết

Để giải quyết sự đa dạng về cấu trúc dữ liệu giữa các hệ thống, trực tích hợp xây dựng một bộ đặc tả dữ liệu thống nhất và lưu trữ trong MongoDB. Bộ đặc tả này quy định rõ tên trường, kiểu dữ liệu, các ràng buộc bắt buộc, giá trị mặc định và mối liên hệ giữa các trường dữ liệu.

Khi một đơn vị gửi dữ liệu lên trực tích hợp, dữ liệu đó bắt buộc phải tuân thủ đúng cấu trúc đã được định nghĩa trước. Nếu dữ liệu không phù hợp, hệ thống sẽ từ chối tiếp nhận và trả về thông báo lỗi tương ứng. Nhờ đó, các phòng ban buộc phải sử dụng cùng một chuẩn dữ liệu, giúp hạn chế sự phân mảnh giữa các hệ thống, đồng thời hỗ trợ việc kiểm tra và phát hiện lỗi dễ dàng hơn.

Listing 1: Ví dụ đặc tả dữ liệu tổng quát trong MongoDB

```
1 {  
2   "tableName": "ten_bang",  
3   "dataFrom": "ten_he_thong",  
4   "dataFromApi": "/duong_dan_api",  
5   "dataFromMethod": "GET",  
6   "description": "Mo ta bang du lieu",  
7  
8   "details": [  
9     {
```

```
10     "name": "ten_truong",
11     "type": "kieu_du_lieu",
12     "length": 10
13 },
14 {
15     "name": "trang_thai",
16     "type": "boolean",
17     "length": null
18 }
19 ],
20
21 "primaryKey": ["khoa_chinh"],
22
23 "fieldsCount": 2,
24 "recordsCount": 100,
25
26 "status": "stable",
27 "syncStrategy": "overwrite"
28 }
```

Hệ thống hiện thực chiến lược phòng thủ đa tầng được chia thành hai lớp xử lý tương ứng với hai thành phần cốt lõi của luồng xử lý.

3.2.1 Tầng tiếp nhận dữ liệu — DataIntegrationService

Tầng tiếp nhận chịu trách nhiệm kiểm tra và lọc dữ liệu ngay tại điểm giao tiếp, trước khi bất kỳ bản ghi nào được đưa vào các bước xử lý sâu hơn.

Kiểm tra cấu trúc phản hồi API (Fail-Fast Validation). Mọi phản hồi từ hệ thống nguồn đều được kiểm tra bốn điều kiện bắt buộc ngay sau khi nhận:

```
if (!responseData
    || responseData.success !== true
    || !responseData.payload
    || !Array.isArray(responseData.payload)) {
    throw new Error('[API CONTRACT VIOLATION]...');
}
```



Chiến lược thất bại nhanh (*fail-fast*) đảm bảo rằng dữ liệu sai định dạng bị từ chối ngay tại điểm tiếp nhận, không lan truyền vào các tầng xử lý phía sau. Bảng dữ liệu bị lỗi được ghi nhận và hệ thống tiếp tục xử lý các bảng kế tiếp.

Bộ lọc trạng thái lược đồ (Schema Status Guard). Trước mỗi phiên đồng bộ, hệ thống kiểm tra trạng thái lược đồ của từng bảng trong MongoDB. Chỉ các bảng có trạng thái **stable** mới được phép đồng bộ. Các bảng ở trạng thái **changed** hoặc **new** sẽ bị bỏ qua và cảnh báo được ghi vào nhật ký, chờ quản trị viên xem xét và phê duyệt thủ công.

Bộ lọc trang lệch (Page Mismatch Guard). Sau mỗi lần nhận phản hồi phân trang, hệ thống so sánh số trang thực tế trong siêu dữ liệu (**metadata**) với số trang đang được yêu cầu. Nếu phát hiện sai lệch, vòng lặp phân trang bị dừng ngay lập tức để tránh ghi đè dữ liệu sai:

```
if (meta.currentPage && meta.currentPage !== currentPage) {  
  // [PAGE MISMATCH] Expected page P, got Q. Skipping.  
  break;  
}
```

Bộ lọc dữ liệu rỗng. Nếu hệ thống nguồn trả về mảng **payload** rỗng trước khi đạt tới tổng số trang đã khai báo, vòng lặp phân trang sẽ dừng sớm để tránh lặp vô hạn do lỗi logic phân trang từ phía nguồn.

3.2.2 Tầng đồng bộ — SyncEngineService

Tầng đồng bộ xử lý từng lô dữ liệu đã qua tầng tiếp nhận, áp dụng các cơ chế làm sạch và chuẩn hóa trước khi ghi vào PostgreSQL.

Lọc trường dữ liệu không xác định. Hệ thống truy vấn Prisma DMMF (*Data Model Meta Format*) tại thời gian thực thi (runtime) để lấy danh sách các trường hợp lệ của từng bảng. Bất kỳ trường nào từ nguồn không nằm trong danh sách này đều bị loại bỏ trước khi tiến hành ghi cơ sở dữ liệu:

```
const validFields = this.getValidScalarFields(modelName);  
const filteredRecord = Object.fromEntries(  
  Object.entries(record).filter(([k]) => validFields.has(k))  
);
```

Cơ chế này loại bỏ hoàn toàn lỗi “*Unknown field*” của Prisma mà không cần can thiệp thủ công khi hệ thống nguồn tùy tiện thêm cột mới.

Loại bỏ trùng lặp khóa chính. Trước khi `upsert`, toàn bộ lô dữ liệu được xử lý qua hàm loại bỏ trùng lặp sử dụng cấu trúc `Map` với từ khóa là giá trị khóa chính. Bản ghi xuất hiện sau cùng với cùng khóa chính sẽ được giữ lại. Các bản ghi có khóa chính `null` hoặc `undefined` không bị loại bỏ tại bước này mà được chuyển tiếp sang vòng lặp `upsert` để cố ý tạo ra lỗi và được ghi nhận vào nhật ký lỗi.

Chuẩn hóa kiểu dữ liệu. Dựa trên siêu dữ liệu từ Prisma DMMF, hệ thống tự động chuyển đổi các giá trị sai kiểu từ nguồn về kiểu dữ liệu đúng:

Bảng 3.2: Quy tắc chuẩn hóa kiểu dữ liệu

Kiểu Prisma	Nguồn gửi	Sau chuẩn hóa
<code>DateTime</code>	"1990-06-15T00:00:00Z"	Đối tượng <code>Date</code>
<code>DateTime</code>	"not-a-date"	<code>null</code>
<code>Int</code>	"701"	701
<code>Float</code>	"3.14"	3.14
<code>Boolean</code>	"true" / "1"	<code>true</code>
<code>Boolean</code>	"false"	<code>false</code>

Cập nhật bất biến (Idempotent Upsert). Thay vì thực hiện `INSERT` hoặc `UPDATE` riêng lẻ, hệ thống sử dụng cơ chế `upsert` của Prisma cho mọi bản ghi. Với cơ chế này, việc chạy lại luồng xử lý nhiều lần trên cùng một tập dữ liệu không tạo ra các bản ghi trùng lặp mà chỉ cập nhật thông tin mới nhất, đảm bảo tính bất biến của hệ thống.

Nhật ký lỗi (Dead Letter Log). Mọi bản ghi gây lỗi trong quá trình `upsert` — dù là thiếu khóa chính, vi phạm ràng buộc `VARCHAR`, hay lỗi cơ sở dữ liệu khác — đều được bắt giữ riêng lẻ trong khối `try/catch` và ghi vào mảng `errors` của nhật ký sự kiện trong MongoDB. Luồng xử lý không bị hủy; các bản ghi hợp lệ còn lại trong cùng lô vẫn tiếp tục được đồng bộ. Quản trị viên có thể truy vết chính xác từng bản ghi lỗi thông qua giá trị khóa chính và thông báo lỗi được lưu kèm.

3.2.3 Tổng hợp cơ chế phòng thủ

Bảng 3.3: Tổng hợp các cơ chế phòng thủ xung đột dữ liệu

Dạng xung đột	Vị trí xử lý	Cơ chế giải quyết
Vi phạm cấu trúc phản hồi	DataIntegration	Fail-fast validation, từ chối ngay tại đầu vào
Lược đồ thay đổi chưa duyệt	DataIntegration	Bộ lọc trạng thái lược đồ (<code>status = stable</code>)
Trang phân trang lệch	DataIntegration	Bộ lọc trang lệch, dừng vòng lặp
Dữ liệu rỗng bất thường	DataIntegration	Bộ lọc dữ liệu rỗng, dừng vòng lặp
Trường lạ từ nguồn	SyncEngineService	Lọc trường hợp lệ qua Prisma DMMF
Trùng khóa chính trong lô	SyncEngineService	Loại bỏ trùng lặp bằng cấu trúc Map
Sai kiểu dữ liệu	SyncEngineService	Chuẩn hóa kiểu dữ liệu qua Prisma DMMF
Chạy lại luồng xử lý	SyncEngineService	Cơ chế <code>upsert</code> bất biến
Lỗi bản ghi đơn lẻ	SyncEngineService	Lưu nhật ký lỗi (<i>Dead Letter Log</i>) vào MongoDB

Dựa trên các nghiên cứu lý thuyết ở chương trước, chương này trình bày kiến trúc thực tế đã được hiện thực hóa cho hệ thống Trục tích hợp dữ liệu. Kiến trúc được điều chỉnh so với đề xuất ban đầu để phù hợp với phạm vi dự án capstone và các ràng buộc kỹ thuật thực tế.

4 Thiết kế kiến trúc

4.1 Yêu Cầu Chức Năng và Phi Chức Năng

Yêu Cầu Chức Năng

- **Tích hợp dữ liệu từ hệ thống nguồn:** Hệ thống kết nối và trích xuất dữ liệu từ API của hệ thống nguồn thông qua giao thức HTTP/REST, hỗ trợ phân trang và

xử lý dữ liệu theo lô.

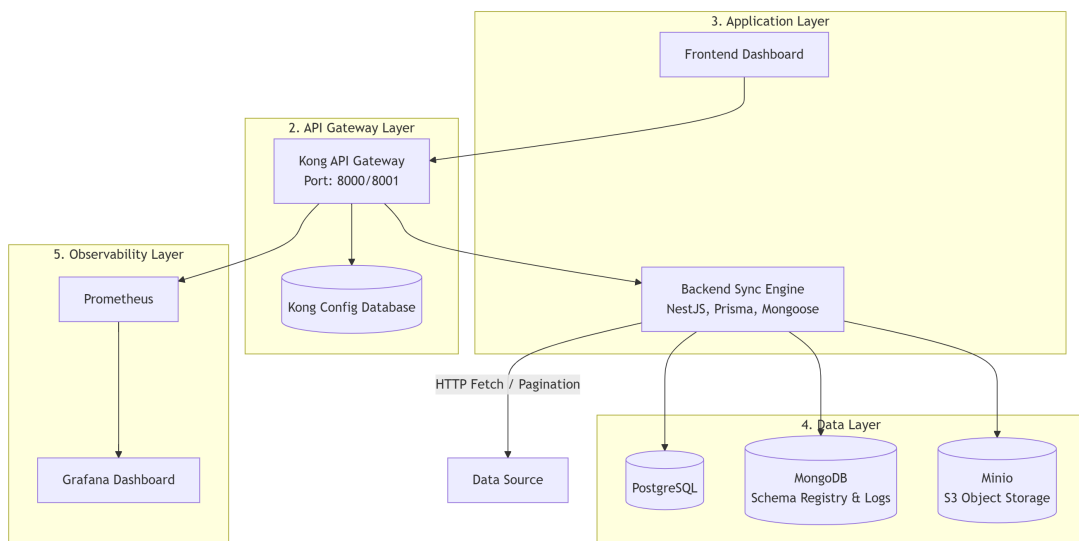
- **Đồng bộ theo nhiều chiến lược:** Hỗ trợ ba chiến lược đồng bộ — **upsert** (thêm mới hoặc cập nhật), **incremental** (chỉ xử lý bản ghi thay đổi theo **updatedAt**), và **overwrite** (xóa toàn bộ rồi ghi lại) — được cấu hình riêng cho từng bảng thông qua Schema Registry.
- **Lập lịch đồng bộ tự động:** Cho phép tạo, cấu hình và kích hoạt/vô hiệu hóa các tác vụ đồng bộ định kỳ thông qua biểu thức cron. Lịch được lưu trong MongoDB và nạp động vào bộ lập lịch khi khởi động.
- **Quản lý schema:** Tự động phát hiện thay đổi cấu trúc dữ liệu tại hệ thống nguồn thông qua so sánh hash. Khi schema thay đổi, bảng bị khóa khỏi chu kỳ đồng bộ cho đến khi quản trị viên xem xét và phê duyệt.
- **Sao lưu và phục hồi:** Tự động tạo snapshot trước mỗi lần sync, sau mỗi thay đổi schema và theo lịch hàng tuần. Hỗ trợ restore về bất kỳ điểm nào trong lịch sử backup.
- **Xác thực và phân quyền:** Mọi yêu cầu API đều phải qua xác thực JWT. Phân quyền theo vai trò kết hợp phạm vi bảng dữ liệu cụ thể.
- **Giám sát và cảnh báo:** Thu thập metrics qua Prometheus, hiển thị trên Grafana, và gửi email cảnh báo khi tác vụ đồng bộ thất bại.

Yêu Cầu Phi Chức Năng

- **Tính bảo mật:** Mật khẩu được băm bằng **bcrypt**. Token JWT ký bằng khóa bí mật. Toàn bộ giao tiếp qua Kong API Gateway với kiểm soát CORS và xác thực token tập trung.
- **Tính bền vững dữ liệu:** Backup đa lớp (MinIO nội bộ + AWS S3 đám mây) đảm bảo dữ liệu không bị mất ngay cả khi hạ tầng on-premise gặp sự cố.
- **Khả năng quan sát:** Toàn bộ sự kiện đồng bộ, lỗi và kết quả được ghi vào Event Log (MongoDB). Metrics được thu thập và hiển thị theo thời gian thực trên Grafana.
- **Khả năng bảo trì:** Backend được tổ chức theo mô hình module hóa của NestJS — mỗi module có trách nhiệm duy nhất, dễ dàng mở rộng hoặc thay thế độc lập.
- **Tính sẵn sàng:** Tất cả container được cấu hình **restart: always** — tự động khởi động lại sau sự cố hoặc khi máy chủ reboot.

4.2 Sơ Đồ Kiến Trúc Tổng Thể

So với kiến trúc lý thuyết API-led Connectivity ba lớp (System API, Process API, Experience API) ban đầu, kiến trúc thực tế được hợp nhất thành một **backend monolith** theo mô hình module hóa của NestJS. Quyết định này xuất phát từ phạm vi dự án capstone: số lượng nguồn dữ liệu, lưu lượng và team size chưa đủ để biện minh cho chi phí vận hành của kiến trúc microservice đầy đủ.



Hình 4.1: Luồng cập lại mã truy cập

4.3 Giải Thích Chi Tiết Các Thành Phần

4.3.1 Admin Dashboard (Next.js Frontend)

Giao diện quản trị được xây dựng bằng Next.js, đóng vai trò là điểm tương tác duy nhất cho quản trị viên. Các chức năng chính bao gồm: quản lý người dùng và phân quyền, theo dõi lịch sử đồng bộ qua Event Log, kích hoạt và giám sát tác vụ đồng bộ, quản lý Schema Registry, và thao tác với hệ thống backup.

Frontend giao tiếp hoàn toàn qua Kong API Gateway, không gọi trực tiếp vào backend. Biến `NEXT_PUBLIC_KONG_URL` được nhúng vào bundle tại thời điểm build — thay đổi địa chỉ yêu cầu build lại container.

4.3.2 Kong API Gateway

Kong là cổng vào duy nhất cho mọi yêu cầu từ frontend. Cấu hình được quản lý khai báo qua file `kong.yml` và đồng bộ vào Kong runtime thông qua công cụ `deck`. Các trách nhiệm của Kong:

- **Định tuyến:** Ánh xạ các đường dẫn (`/auth`, `/users`, `/integration`, v.v.) tới backend tại `http://backend:4000/api/v1`.
- **CORS:** Plugin CORS kiểm soát các origin được phép gọi API — quan trọng khi triển khai trên EC2 với Public IP cụ thể.
- **Metrics:** Plugin Prometheus thu thập số liệu về lưu lượng và độ trễ của từng route.

4.3.3 NestJS Backend (Monolith)

Backend là một ứng dụng NestJS duy nhất, được tổ chức thành các module độc lập. Mỗi module đóng gói controller, service và các dependency của riêng nó.

Auth Module và Users Module Hai module này hiện thực hóa cơ chế xác thực và phân quyền. Xác thực dựa trên JWT với Passport.js. Phân quyền theo mô hình vai trò hai cấp: vai trò cứng (`admin`, `reader`, `writer`) kết hợp phạm vi bảng dữ liệu linh hoạt qua `roleSettings` dạng regex pattern. Chi tiết được trình bày tại Chương ??.

Schema Registry Module Lưu trữ metadata của từng bảng dữ liệu trong MongoDB, bao gồm: chiến lược đồng bộ (`syncStrategy`), khóa chính, danh sách trường, hash của cấu trúc hiện tại, trạng thái (`stable` / `changed` / `new`), và thời điểm đồng bộ gần nhất (`lastSyncTime`).

Mỗi lần pipeline chạy, Schema Registry so sánh hash cấu trúc trả về từ API nguồn với hash đang lưu. Nếu phát hiện thay đổi, bảng bị đánh dấu **changed**, emit event **schema.changed** để kích hoạt backup kiểm toán, và bị loại khỏi chu kỳ sync cho đến khi admin xem xét.

Data Integration Module Đây là module trung tâm, hiện thực hóa toàn bộ pipeline ETL. Khi được kích hoạt (thủ công hoặc qua lịch), module thực hiện tuần tự:

1. Lấy danh sách bảng **stable** từ Schema Registry.
2. Với mỗi bảng: tạo pre-sync backup, sau đó fetch dữ liệu từ API nguồn theo chiến lược đã cấu hình.
3. Ghi dữ liệu vào PostgreSQL thông qua Sync Engine.
4. Phát hiện orphan records (bản ghi tồn tại ở destination nhưng không còn ở source) nếu điều kiện cho phép.
5. Ghi kết quả vào Event Log và gửi email thông báo qua Notification Module.

Module hỗ trợ hai loại pipeline: **Full Sync** (toàn bộ bảng stable) và **Custom Sync** (danh sách bảng do admin chỉ định).

Job Scheduler Module Quản lý các tác vụ đồng bộ định kỳ. Mỗi tác vụ được lưu trong MongoDB với biểu thức cron, loại job (**FULL_SYNC**), trạng thái (**active/inactive**) và thời điểm chạy gần nhất.

Khi backend khởi động, toàn bộ tác vụ active được đăng ký động vào **SchedulerRegistry** của NestJS. Admin có thể tạo, sửa, bật/tắt và kích hoạt thủ công tác vụ qua API mà không cần restart server.

Backup Module Quản lý toàn bộ vòng đời backup: tạo snapshot (4 trigger), liệt kê, tải về, xóa và restore. Backup được lưu trên MinIO theo cấu trúc **{trigger}/{tableName}/{timestamp}.json**. Admin có thể đồng bộ thủ công lên AWS S3 để lưu trữ vĩnh viễn. Chi tiết được trình bày tại Chương ??.

Event Log Module Ghi nhận mọi sự kiện đáng chú ý vào MongoDB: bắt đầu/kết thúc pipeline, thành công/thất bại của từng bảng, phát hiện orphan, restore backup. Event Log là nguồn dữ liệu chính để admin theo dõi lịch sử hoạt động của hệ thống.



Notification Module Gửi email cảnh báo qua SMTP (Gmail) khi tác vụ đồng bộ thất bại hoặc hoàn thành. Email chứa thông tin: tên tác vụ, thời gian, thời lượng thực thi, và chi tiết lỗi (nếu có). Module được đánh dấu `@Global` để có thể inject vào bất kỳ module nào mà không cần khai báo lại.

4.3.4 Tầng Lưu Trữ

PostgreSQL (qua Prisma ORM) Cơ sở dữ liệu quan hệ chính, lưu trữ toàn bộ dữ liệu nghiệp vụ đã được đồng bộ từ các hệ thống nguồn, cùng với thông tin người dùng và cấu hình retention policy. Prisma ORM đảm bảo type-safety và quản lý migration có kiểm soát.

MongoDB (qua Mongoose) Cơ sở dữ liệu document, lưu trữ ba loại dữ liệu vận hành: Schema Registry (metadata và cấu hình đồng bộ của từng bảng), Event Log (nhật ký hoạt động), và Scheduled Jobs (cấu hình tác vụ định kỳ). MongoDB Atlas được sử dụng dưới dạng dịch vụ cloud — không cần triển khai và quản lý MongoDB server.

MinIO và AWS S3 Lưu trữ backup theo kiến trúc hai tầng: MinIO (self-hosted, độ trễ thấp) và AWS S3 (cloud, lưu trữ vĩnh viễn). Chi tiết tại Chương ??.

4.3.5 Hệ Thống Nguồn Dữ Liệu

Trong phạm vi dự án, hệ thống nguồn là một REST API cung cấp dữ liệu sinh viên qua giao thức HTTP. API trả về dữ liệu theo định dạng chuẩn:

```
{
  "success": true,
  "metadata": { "totalRecords": 68411, "totalPages": 14 },
  "payload": [ { "id": 1, ... } ]
}
```

Hệ thống tích hợp đóng vai trò consumer thuần túy — chỉ đọc dữ liệu qua API, không kết nối trực tiếp vào cơ sở dữ liệu nguồn.

4.3.6 Giám Sát: Prometheus và Grafana

Prometheus thu thập metrics từ Kong (qua plugin) và backend (qua endpoint `/metrics`). Grafana hiển thị các dashboard theo dõi: số lượng request, tỷ lệ lỗi, độ trễ phản hồi, và

trạng thái các container. Grafana được cấu hình gửi cảnh báo qua email khi phát hiện bất thường.

4.4 So Sánh Với Kiến Trúc Lý Thuyết Ban Đầu

Bảng 4.1: So sánh kiến trúc lý thuyết và thực tế

Khía cạnh	Lý thuyết	Thực tế
Mô hình	API-led (3 lớp riêng biệt)	Monolith module hóa (NestJS)
Data Sharing Server	Server riêng (Process + Experience API)	Tích hợp vào backend monolith
Data Integration Server	Server riêng (System API)	Module trong cùng backend
Phân quyền	RBAC đầy đủ (5 bảng)	Role enum + roleSettings JSON (regex)
Nguồn dữ liệu	Nhiều CSDL (Đào tạo, Tài chính, Nhân sự)	Một REST API nguồn
Backup	Không đề cập	MinIO + AWS S3, 4 trigger
Schema quản lý	Không đề cập	Schema Registry (MongoDB)
Cảnh báo	Không đề cập	Email alert qua SMTP
Triển khai	Không đề cập	Docker Compose trên AWS EC2

Sự khác biệt lớn nhất so với thiết kế lý thuyết là việc hợp nhất Data Sharing Server và Data Integration Server thành một backend duy nhất. Quyết định này được đưa ra vì: (1) số lượng nguồn dữ liệu và lưu lượng hiện tại chưa đủ để biện minh cho độ phức tạp vận hành của microservices; (2) kiến trúc module hóa của NestJS đã cung cấp sự tách biệt trách nhiệm đủ tốt trong phạm vi một process duy nhất; (3) dễ dàng di chuyển sang microservices trong tương lai bằng cách tách từng module thành service độc lập khi cần.

5 Cơ chế xác thực và phân quyền

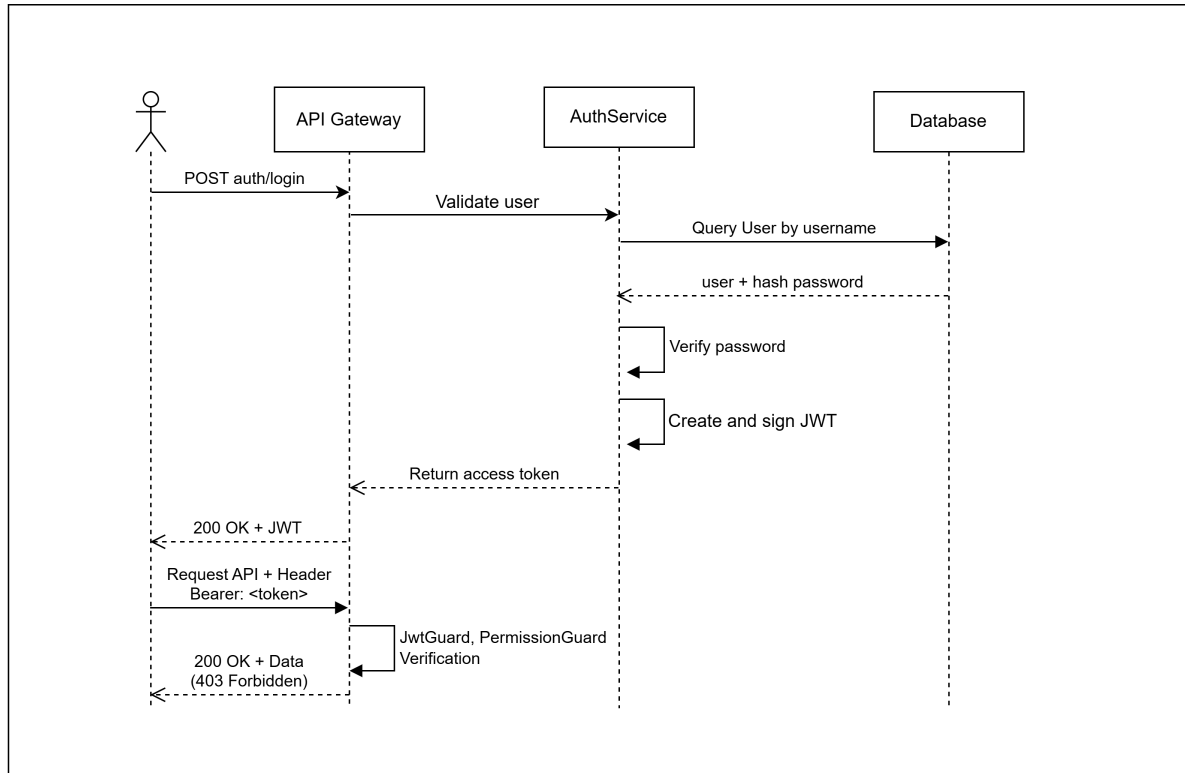
5.1 Luồng xác thực

Xác thực là quá trình kiểm tra và xác minh danh tính của người dùng trước khi cho phép truy cập vào hệ thống. Trục tích hợp dữ liệu sử dụng JSON Web Token (JWT) kết hợp với thư viện Passport.js để thực hiện cơ chế xác thực phi trạng thái (*stateless*).

Quy trình xác thực diễn ra theo các bước sau:

1. Người dùng gửi yêu cầu đăng nhập kèm theo tên đăng nhập và mật khẩu đến điểm cuối API `POST /auth/login`.
2. Dịch vụ xác thực truy vấn bảng `users` trong PostgreSQL để tìm người dùng theo tên đăng nhập. Mật khẩu được xác minh thông qua hàm `bcrypt.compare` bằng cách đối chiếu với giá trị băm đã lưu trong cột `passwordHash`.
3. Nếu thông tin hợp lệ, hệ thống khởi tạo một mã truy cập JWT với phần dữ liệu tải trọng (payload) chứa các thông tin sau:
 - `sub`: Định danh người dùng (ID).
 - `username`: Tên đăng nhập.
 - `role`: Vai trò của người dùng (`admin` | `reader` | `writer`).
 - `roleSettings`: Cấu hình phạm vi quyền hạn dưới định dạng JSON (được trình bày chi tiết tại Mục 5.2).
4. Mã truy cập được ký số bằng khóa bí mật `JWT_SECRET` và có thời gian tồn tại được quy định thông qua biến môi trường `JWT_EXPIRES_IN`.
5. Sau khi nhận được mã truy cập, người dùng đính kèm mã này vào mọi yêu cầu truy xuất tài nguyên tiếp theo. Hệ thống hỗ trợ hai hình thức truyền mã truy cập:
 - Qua Tiêu đề HTTP (HTTP Header):
`Authorization: Bearer <token>`
 - Qua tham số truy vấn (URL Query - áp dụng cho các kết nối luồng dữ liệu liên tục như SSE):
`GET /integration/stream-logs?token=<token>`

Việc sử dụng JWT giúp hệ thống tuân thủ hoàn toàn kiến trúc phi trạng thái — máy chủ không cần lưu trữ thông tin phiên làm việc (session); mọi thông tin cần thiết để xác thực và phân quyền đều được nhúng gọn bên trong mã thông báo.



Hình 5.1: Luồng xác thực và kiểm tra quyền truy cập

5.2 Luồng phân quyền

Sau khi hệ thống xác thực người dùng thành công, luồng yêu cầu tiếp tục đi qua hai lớp bảo vệ độc lập để thực hiện kiểm tra quyền truy cập một cách tuần tự.

5.2.1 Lớp 1 — Kiểm tra vai trò (RolesGuard)

Hệ thống định nghĩa ba vai trò cố định, được lưu trực tiếp trên bảng **users**:

- **admin**: Quản trị viên hệ thống, có toàn quyền truy cập tài nguyên và được đặc cách bỏ qua mọi bước kiểm tra phân quyền phía sau.
- **reader**: Người dùng chỉ có quyền đọc dữ liệu trong phạm vi được chỉ định.
- **writer**: Người dùng có quyền cả đọc và ghi dữ liệu trong phạm vi được chỉ định.

`RolesGuard` chịu trách nhiệm đọc danh sách vai trò cho phép từ bộ trang trí (*decorator*) `@Roles()` gắn trên các trình điều khiển (controller) hoặc từng điểm cuối cụ thể. Sau đó, hệ thống đối chiếu với trường `role` lấy từ tải trọng JWT. Nếu không khớp, yêu cầu sẽ bị từ chối ngay lập tức với mã lỗi 403 Forbidden.

5.2.2 Lớp 2 — Kiểm tra phạm vi bảng (PermissionsGuard)

Ngoài vai trò, mỗi người dùng (ngoại trừ `admin`) được cấp một cấu hình `roleSettings` lưu dưới định dạng JSON trong cơ sở dữ liệu:

Listing 2: Cấu trúc cấu hình `roleSettings`

```
1 {  
2   "writeScopes": ["^tcns_.*", "^dm_gioi_tinh$"],  
3   "readScopes":  ["^nguoi_hoc$", "^dm_.*"]  
4 }
```

Mỗi phần tử trong mảng `writeScopes` và `readScopes` là một biểu thức chính quy (*regular expression*) dùng để so khớp với tên bảng được truyền qua tham số động `:table` của URL yêu cầu. Logic kiểm tra được thực thi như sau:

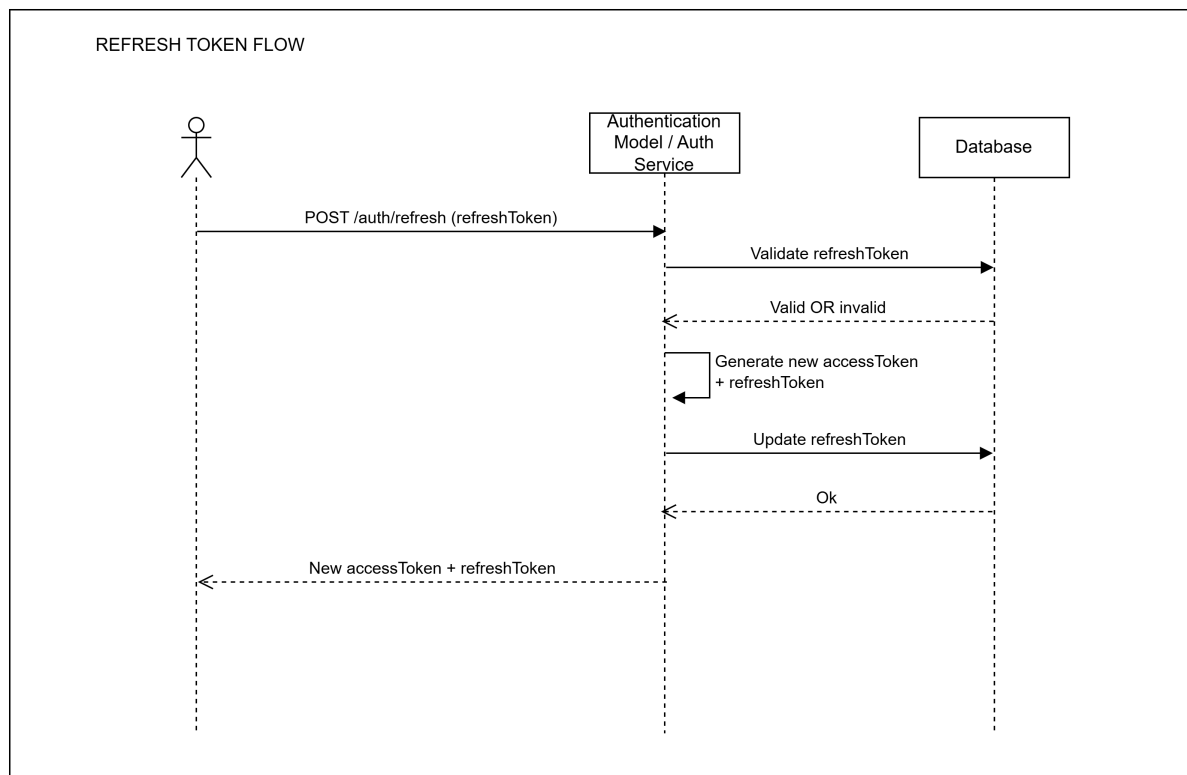
- **Admin:** Bỏ qua hoàn toàn bước kiểm tra này.
- **Thao tác đọc** (Phương thức GET):
 - **reader:** Tên bảng phải khớp với ít nhất một biểu thức trong `readScopes`.
 - **writer:** Tên bảng phải khớp với ít nhất một biểu thức trong `readScopes` **hoặc** `writeScopes`.
- **Thao tác ghi** (Phương thức POST, PUT, PATCH, DELETE): Chỉ tài khoản **writer** được phép thực hiện, và tên bảng bắt buộc phải khớp với ít nhất một biểu thức trong `writeScopes`.

Việc ứng dụng biểu thức chính quy mang lại tính linh hoạt rất cao trong cấu hình. Ví dụ: biểu thức `^tcns_.*` cấp quyền truy cập hàng loạt cho toàn bộ các bảng có tiền tố `tcns_`, trong khi biểu thức `^dm_gioi_tinh$` giới hạn quyền truy cập chặt chẽ vào đúng một bảng duy nhất.

5.3 Cơ chế cấp lại mã truy cập

Khi mã truy cập ngắn hạn hết hiệu lực, người dùng chỉ cần gửi lại mã cũ đến điểm cuối `POST /auth/refresh-token`. Hệ thống sẽ xác minh chữ ký và tính toàn vẹn của mã thông qua phương thức `jwtService.verify()`, sau đó trích xuất dữ liệu gốc để ký lại và phát hành một mã truy cập hoàn toàn mới.

Cơ chế này giữ được sự tối giản về mặt logic và không yêu cầu phải lưu trữ thêm *Refresh Token* trong cơ sở dữ liệu, đáp ứng tốt yêu cầu về mô hình phi trạng thái của toàn bộ hệ thống.



Hình 5.2: Luồng xử lý cấp lại mã truy cập (Refresh Token)

5.4 Cấu trúc cơ sở dữ liệu phục vụ xác thực và phân quyền

Khác với mô hình kiểm soát truy cập dựa trên vai trò truyền thống (RBAC - Role-Based Access Control) thường đòi hỏi nhiều bảng trung gian phức tạp, hệ thống trực tích hợp đã hợp nhất toàn bộ thông tin xác thực và phân quyền vào một bảng duy nhất:

- **users:** Lưu trữ thông tin định danh bao gồm tên đăng nhập, mật khẩu đã băm



(`passwordHash`), họ tên, thư điện tử, kết hợp với vai trò hệ thống (`role`) và cấu hình phạm vi quyền (`roleSettings`).

Lối thiết kế này đạt được sự cân bằng tối ưu giữa tính đơn giản và độ linh hoạt: trường vai trò kiểu liệt kê (enum) đảm bảo tính nhất quán cho các thao tác cấp cao, trong khi trường `roleSettings` lưu dạng JSON cho phép tùy biến phạm vi quyền chi tiết đến từng bảng cụ thể mà không cần phải thực hiện di cư dữ liệu (migration) làm thay đổi cấu trúc bảng.

6 Luồng xử lý dữ liệu

6.1 Chiến lược đồng bộ dữ liệu và cơ chế ghi vào cơ sở dữ liệu

6.1.1 Tổng quan

Sau khi dữ liệu được lấy về từ API nguồn, hệ thống cần quyết định *cách thức ghi* vào cơ sở dữ liệu đích thông qua Prisma ORM. Thay vì áp dụng một phương thức duy nhất cho mọi bảng, hệ thống thiết kế ba chiến lược riêng biệt, mỗi chiến lược phù hợp với đặc thù của từng loại dữ liệu:

- **upsert** — chiến lược mặc định, phù hợp với các bảng nghiệp vụ lớn, dữ liệu có sự thay đổi thường xuyên.
- **incremental** — chiến lược đồng bộ tăng tiến, giúp giảm tải băng thông mạng và cơ sở dữ liệu bằng cách chỉ yêu cầu và ghi nhận những bản ghi có sự thay đổi kể từ lần đồng bộ gần nhất.
- **overwrite** — chiến lược ghi đè toàn bộ, phù hợp với các bảng danh mục nhỏ, cần đảm bảo hệ thống đích hoàn toàn đồng nhất 100% với hệ thống nguồn.

Chiến lược được cấu hình tĩnh trong *Schema Registry* (MongoDB) tại trường `syncStrategy`, và được áp dụng tự động tại hàm `syncOneTable()` trong mã nguồn. Thứ tự ưu tiên khi chọn chiến lược diễn ra như sau:

`schema.syncStrategy` $\xrightarrow{\text{nếu không có}}$ `upsert` (mặc định)

Mọi thay đổi về chiến lược đều được áp dụng ngay ở lần đồng bộ tiếp theo mà không cần khởi động lại dịch vụ, nhờ vào cơ chế truy vấn Schema Registry tại thời gian thực thi.

6.1.2 Quy trình chung: Phân trang và tải dữ liệu

Bất kể áp dụng chiến lược nào, hệ thống đều thực hiện **cơ chế phân trang** khi gọi API nguồn, với kích thước mỗi trang cố định là 5.000 bản ghi (`BATCH_LIMIT = 5000`). URL truy vấn được tạo động theo biểu mẫu:

```
{baseUrl}{dataFromApi}?page={currentPage}&limit=5000
[&accessToken={token}]
[&updatedAt={lastSyncTime}]    + chỉ áp dụng khi incremental
```



Hệ thống tiến hành đọc giá trị `metadata.totalPages` từ phản hồi của trang đầu tiên để xác định tổng số trang cần duyệt, sau đó lặp cho đến khi tải hết toàn bộ. Mỗi lần tải đều được bao bọc trong cơ chế **thử lại (retry)** với tối đa 3 lần thao tác nhằm đối phó với lỗi đường truyền mạng.

Hình mô tả luồng xử lý tổng quát của một bảng dữ liệu, từ thời điểm bắt đầu cho đến khi cập nhật trường `lastSyncTime`.

6.1.3 Chiến lược upsert — Mặc định

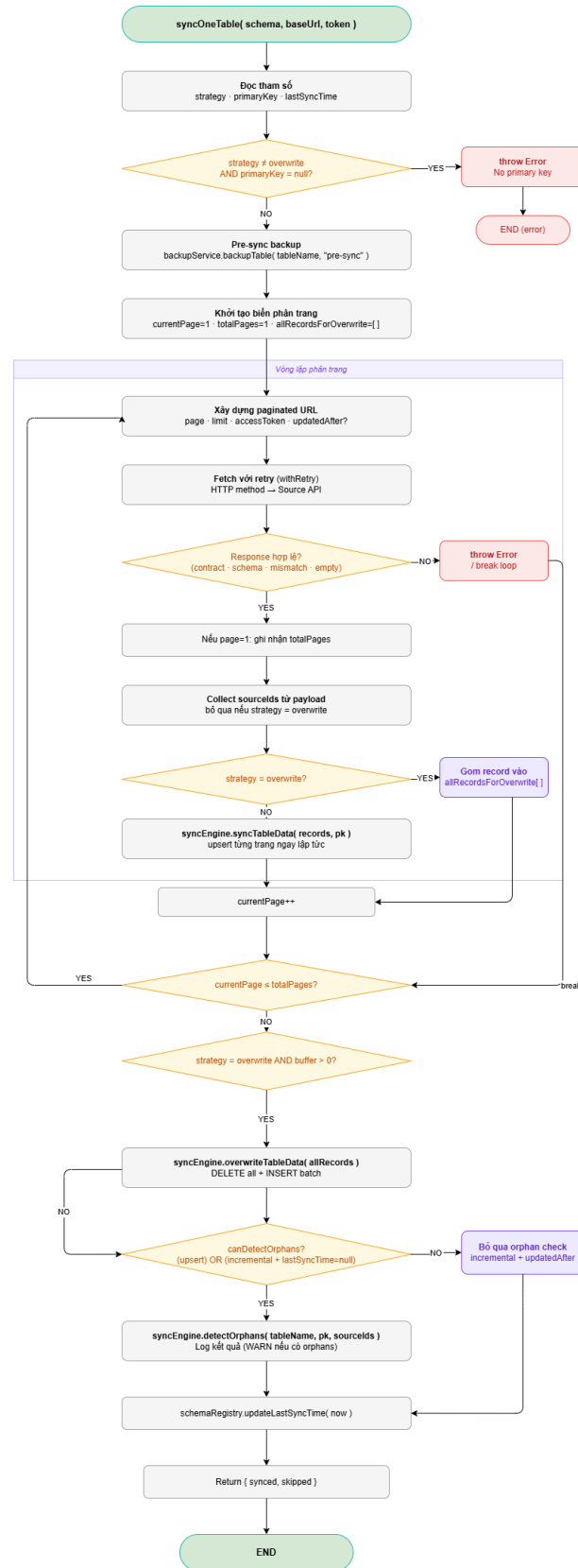
Nguyên lý hoạt động Đây là chiến lược **an toàn và mang tính tổng quát** cao nhất. Với mỗi trang dữ liệu nhận được, hệ thống sẽ tiến hành **upsert** toàn bộ các bản ghi vào PostgreSQL thông qua Prisma:

- Nếu bản ghi chứa **primaryKey** đã tồn tại → thực hiện lệnh **UPDATE** cho các trường còn lại.
- Nếu bản ghi chưa tồn tại → thực hiện lệnh **INSERT** bản ghi mới.

Thao tác này đảm bảo tính **bất biến (idempotent)**: hệ thống có thể chạy lại nhiều lần với cùng một tập dữ liệu đầu vào mà vẫn cho ra cùng một kết quả cuối cùng, không gây ra hiện tượng trùng lặp hay mất mát dữ liệu.

Chi tiết luồng ghi cơ sở dữ liệu Trước khi gọi lệnh **upsert**, mỗi lô bản ghi bắt buộc phải vượt qua bốn lớp tiền xử lý:

1. **Lọc trường dữ liệu (DB Safety Net)** — Loại bỏ các trường không nằm trong định nghĩa lược đồ Prisma. Điều này giúp ngăn chặn lỗi do hệ thống nguồn tự ý bổ sung cột mới mà chưa được thực thi migration trên PostgreSQL.
2. **Loại bỏ trùng lặp (Deduplication)** — Nếu dữ liệu nguồn trả về nhiều bản ghi có cùng khóa chính trong cùng một trang, hệ thống chỉ giữ lại bản ghi *xuất hiện sau cùng*. Các bản ghi có khóa chính bị null không bị loại bỏ ở bước này, mà sẽ được giữ lại để đẩy vào nhật ký lỗi (Dead Letter Log).
3. **Chuẩn hóa kiểu dữ liệu (Type Normalization)** — Chuyển đổi định dạng từ chuẩn JSON về đúng kiểu dữ liệu khai báo trong Prisma:
 - **DateTime**: chuỗi ISO 8601 → Đối tượng **Date**.
 - **Int / Float**: chuỗi chữ số → Kiểu số thực.
 - **Boolean**: "true" / "1" → **true**.
4. **Upsert bất biến và lưu nhật ký lỗi** — Thực hiện lệnh `prisma[model].upsert()` trong vòng lặp. Nếu một bản ghi bị lỗi (vi phạm ràng buộc dữ liệu duy nhất, sai kiểu...), thông báo lỗi được ghi chú vào *Dead Letter Log* trong MongoDB thay vì bắt buộc dừng toàn bộ tiến trình.



Hình 6.1: Cách thức hoạt động của hàm syncOneTable

Sau khi hoàn tất, kết quả tổng quan sẽ được ghi vào nhật ký sự kiện MongoDB với các chỉ số báo cáo chi tiết: tổng số bản ghi cần đồng bộ, số lượng bản ghi bị loại bỏ trùng lặp, số bản ghi thành công/thất bại, và mảng chi tiết chứa nguyên nhân lỗi của từng bản ghi.

6.1.4 Chiến lược incremental — Bộ lọc tại phía máy chủ

Nguyên lý hoạt động Chiến lược incremental tận dụng khả năng **lọc dữ liệu từ phía API nguồn** thông qua việc đính kèm tham số `updatedAtAfter`, yêu cầu nguồn chỉ truyền trả những bản ghi đã có sự thay đổi kể từ lần đồng bộ cuối cùng. Cách tiếp cận này giúp tối ưu hóa hiệu năng trên cả hai khía cạnh:

- **Bảng thông mạng:** Hệ thống nguồn chỉ trả về một tập con các bản ghi thay vì toàn bộ dữ liệu bảng.
- **Tải cơ sở dữ liệu:** Hệ thống đích chỉ cần thực hiện `upsert` đúng số lượng bản ghi thực sự mới hoặc bị thay đổi.

Mốc thời gian dùng để tạo bộ lọc là trường `lastSyncTime` trong tài liệu cấu hình (Schema Registry), được hệ thống cập nhật tự động sau mỗi lần đồng bộ thành công.

Quản lý tham số `updatedAtAfter` Tham số `updatedAtAfter` được gắn trực tiếp vào URL truy vấn khi và chỉ khi chiến lược được đặt là `incremental` và bảng đã được đồng bộ ít nhất một lần trước đó (`lastSyncTime` khác `null`).

Listing 3: Thêm `updatedAtAfter` vào URL phân trang

```
1 const updatedAtParam =  
2   strategy === 'incremental' && lastSyncTime  
3   ? `&updatedAtAfter=${encodeURIComponent(lastSyncTime.  
4     toISOString())}`  
   : '';
```

API nguồn sau khi tiếp nhận tham số này sẽ tiến hành lọc và chỉ trả về các bản ghi thỏa mãn điều kiện `updatedAt > lastSyncTime`. Toàn bộ số lượng bản ghi trả về sẽ được hệ thống tiến hành `upsert` thẳng vào cơ sở dữ liệu mà không cần phải thực hiện bước kiểm tra điều kiện thêm một lần nữa.

Trường hợp đồng bộ lần đầu tiên Khi `lastSyncTime = null` (tức bảng chưa từng được đồng bộ), tham số `updatedAt` sẽ **không được thêm** vào URL truy vấn. Khi đó, API nguồn sẽ trả về toàn bộ dữ liệu hiện có — hành vi này tương đương với phương thức `upsert` (tải toàn phần). Đây là một cơ chế **khởi tạo ban đầu (bootstrapping)** tự động, giúp quản trị viên không cần phải thiết lập cấu hình thủ công cho lần chạy đầu tiên.

Bảng 6.1: Hiệu quả Incremental: Lọc tại đích (BE-side) và lọc tại nguồn (Server-side)

Tiêu chí	Lọc tại hệ thống đích	Lọc tại hệ thống nguồn (Mới)
Dữ liệu qua mạng	Toàn bộ bảng (100%)	Chỉ tải bản ghi có thay đổi
Số trang cần tải	<code>totalPages</code> đầy đủ	Giảm tỷ lệ thuận theo thay đổi
Xử lý tại backend	Lọc <code>updatedAt</code> cho từng dòng	Upsert thẳng vào DB
Lướt ghi vào DB	Chỉ ghi bản ghi thay đổi	Chỉ ghi bản ghi thay đổi
Lợi thế cốt lõi	Giảm thao tác ghi DB	Giảm cả thao tác ghi DB và băng thông mạng

So sánh hiệu năng chiến lược Incremental

6.1.5 Chiến lược overwrite — Xóa và ghi đè toàn bộ

Nguyên lý hoạt động Chiến lược `overwrite` được thiết kế đặc thù cho các **bảng danh mục (dimension tables)** với kích thước nhỏ, nơi có yêu cầu cốt lõi là: hệ thống đích phải *hoàn toàn giống hệt* hệ thống nguồn sau mỗi lần chạy. Thay vì cập nhật riêng lẻ từng dòng, hệ thống thực hiện các bước sau:

1. **Tải toàn bộ** dữ liệu từ nguồn, **lưu đệm toàn bộ vào bộ nhớ RAM** — hệ thống tuyệt đối không thực hiện bất kỳ thao tác ghi cơ sở dữ liệu nào trong giai đoạn phân trang.
2. **Sau khi tải xong tất cả các trang**, hệ thống mở một giao dịch duy nhất (**transaction**) để thực hiện:
 - (a) `deleteMany({})` — xóa trống toàn bộ dữ liệu bảng hiện tại.
 - (b) `createMany(...)` — chèn lại toàn bộ bản ghi mới được tải về.



Toàn bộ quá trình xóa và chèn được bao bọc trong khối `prisma.$transaction()`, đảm bảo **tính nguyên tử (atomicity)**: nếu quá trình chèn dữ liệu thất bại, thao tác xóa bảng trước đó sẽ lập tức được hoàn tác, ngăn ngừa việc bảng dữ liệu rơi vào trạng thái trống rỗng nửa vời.

Lý do lưu đệm toàn bộ dữ liệu trước khi ghi Nếu hệ thống thực hiện thao tác "xóa rồi chèn" tuần tự theo từng trang, bảng dữ liệu đích sẽ bị xóa sạch ngay sau trang đầu tiên và chỉ chứa một lượng nhỏ dữ liệu của trang đó trong khi chờ tải trang tiếp theo. Điều này gây ra **khoảng thời gian gián đoạn (window downtime)**, ảnh hưởng nghiêm trọng đến các truy vấn đọc đang diễn ra đồng thời của các hệ thống khác. Việc lưu đệm toàn bộ dữ liệu vào RAM rồi mới ghi đè trong một giao dịch duy nhất giúp rút ngắn thời gian gián đoạn xuống mức tối thiểu (chỉ bằng với thời gian thực thi của giao dịch cơ sở dữ liệu).

Miễn trừ cơ chế phát hiện bản ghi mồ côi Vì bản chất của `overwrite` là thực hiện `DELETE all` trước khi chèn mới, mọi bản ghi bị xóa ở hệ thống nguồn hiển nhiên cũng sẽ bị xóa ở hệ thống đích. Do đó, hệ thống được lập trình để **bỏ qua hoàn toàn** bước kiểm tra bản ghi mồ côi cho những bảng sử dụng chiến lược này.

6.2 Cơ Chế Retry Trong ETL Pipeline

6.2.1 Đặt Vấn Đề

Trong quá trình đồng bộ dữ liệu từ hệ thống nguồn *myBK* qua REST API, pipeline ETL phải đối mặt với hai nhóm lỗi có bản chất hoàn toàn khác nhau:

- **Lỗi tạm thời (Transient Errors):** Phát sinh do điều kiện môi trường — mạng không ổn định, server nguồn quá tải nhất thời, timeout do latency cao. Những lỗi này *có thể tự khởi* sau một khoảng thời gian ngắn.
- **Lỗi nghiệp vụ (Business Errors):** Phát sinh do vi phạm hợp đồng dữ liệu — payload sai định dạng, trang phân trang không khớp, khóa chính bị thiếu. Những lỗi này *không thay đổi* dù retry bao nhiêu lần, vì nguyên nhân gốc rễ nằm ở phía dữ liệu, không phải môi trường.

Một cơ chế retry áp dụng đồng nhất cho cả hai nhóm sẽ gây ra hai hệ quả tiêu cực: lãng phí tài nguyên khi retry lỗi nghiệp vụ không thể phục hồi, và che khuất các lỗi thực sự cần được xử lý bởi con người.

6.2.2 Thiết Kế Retry Có Chủ Ý

Hệ thống áp dụng nguyên tắc *Selective Retry*: chỉ retry các lỗi tạm thời ở tầng giao tiếp mạng, và để lỗi nghiệp vụ thất bại ngay lập tức (*fail-fast*) để pipeline có thể ghi nhận và xử lý đúng cách.

Phân Loại Lỗi và Quyết Định Retry Bảng 6.2 tổng hợp chính sách retry theo từng tình huống lỗi trong pipeline.

Bảng 6.2: Chính sách retry theo loại lỗi

Tình huống	Retry	Lý do
Network timeout / ECONNREFUSED	Có	Lỗi tạm thời do môi trường mạng. Server nguồn có thể đã phục hồi sau vài giây.
API trả về 5xx (server nguồn lỗi)	Có	Server nguồn có thể đang khởi động lại hoặc quá tải nhất thời. Retry sau 10 giây có khả năng thành công.
[API CONTRACT VIOLATION]	Không	Payload sai cấu trúc (<code>success != true</code> , thiếu <code>payload[]</code>). Retry sẽ nhận lại cùng response sai — không có ý nghĩa.
[SCHEMA CONTRACT VIOLATION]	Không	Payload chứa field chưa định nghĩa trong metadata. Đây là lỗi cấu hình cần admin xử lý thủ công.
[PAGE MISMATCH]	Không	Server nguồn trả về sai số trang phân trang. Retry vẫn nhận trang sai — cần điều tra phía nguồn.
DB constraint violation (upsert)	Không	Record vi phạm ràng buộc cơ sở dữ liệu sẽ tiếp tục thất bại trong mọi lần retry. Chuyển sang <i>Dead Letter Log</i> để xử lý riêng.

Tham Số Retry Pipeline được cấu hình với các tham số sau:

- **Số lần retry tối đa:** 3 lần (tổng cộng 4 lần thử bao gồm lần đầu tiên).
- **Khoảng chờ giữa các lần retry:** 10 giây (fixed delay).
- **Phạm vi áp dụng:** Chỉ tầng HTTP fetch — không áp dụng cho tầng ghi cơ sở dữ liệu.

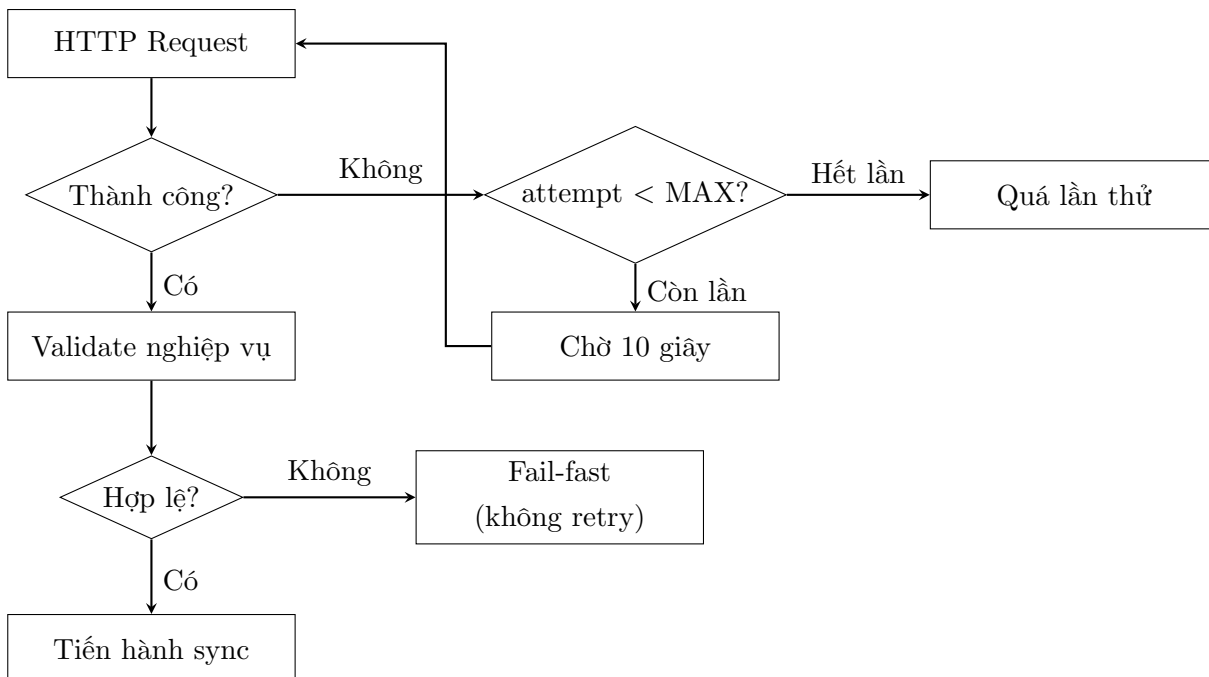
Cài Đặt Retry được đóng gói trong phương thức `withRetry<T>()` — một generic wrapper nhận vào một hàm bất đồng bộ và tên ngữ cảnh để ghi log. Phương thức này

được gọi quanh mỗi HTTP request trong vòng lặp phân trang.

Lỗi nghiệp vụ được ném ra *bên ngoài* `withRetry()` — cụ thể là sau khi response đã được nhận thành công từ mạng — do đó chúng không bị bắt bởi vòng lặp retry

6.2.3 Luồng Xử Lý Khi Retry

Hình 6.2 mô tả luồng quyết định khi một HTTP request thất bại.



Hình 6.2: Luồng quyết định retry trong pipeline ETL

6.2.4 Đánh Giá

Thiết kế retry có chủ ý mang lại các lợi ích sau:

1. **Tăng độ bền:** Pipeline tự phục hồi khi gặp lỗi mạng tạm thời mà không cần can thiệp thủ công, đặc biệt quan trọng trong môi trường ngrok với độ ổn định không cao.
2. **Phát hiện lỗi nhanh (Fail-Fast):** Lỗi nghiệp vụ được báo cáo ngay lập tức thay vì bị che khuất bởi nhiều lần retry vô ích, giúp rút ngắn thời gian phát hiện sự cố từ phía nguồn.



3. **Tiết kiệm tài nguyên:** Không lãng phí 30 giây chờ ($3 \text{ lần} \times 10 \text{ giây}$) cho các lỗi xác định không thể phục hồi.
4. **Log có ngữ cảnh:** Mỗi lần retry được ghi kèm số thứ tự, tên bảng, số trang và thông báo lỗi gốc, giúp vận hành dễ dàng truy vết nguyên nhân.

Hạn chế: Cơ chế hiện tại sử dụng *fixed delay* (10 giây cố định). Trong môi trường production với tải cao, chiến lược *exponential backoff with jitter* sẽ hiệu quả hơn vì tránh được hiện tượng nhiều client retry đồng thời. Đây là hướng cải tiến cho phiên bản tiếp theo.

6.3 Phát hiện bản ghi mồ côi (Orphan Detection)

6.3.1 Đặt vấn đề

Các chiến lược đồng bộ **upsert** và đồng bộ tăng tiến (**incremental**) chỉ xử lý hai chiều biến động của dữ liệu: thêm mới và cập nhật. Chiều thứ ba - xóa bản ghi tại hệ thống nguồn - hoàn toàn nằm ngoài phạm vi xử lý của hai chiến lược này.

Khi một bản ghi bị xóa tại hệ thống nguồn, nó không còn xuất hiện trong phản hồi của API. Tuy nhiên, bản ghi tương ứng tại hệ thống đích (PostgreSQL) vẫn tồn tại nguyên vẹn vì không có lệnh **DELETE** nào được phát ra. Theo thời gian, những bản ghi còn lại này tích lũy ngày càng nhiều, tạo ra sự phân mảnh lớn so với dữ liệu nguồn thực tế.

Định nghĩa chính thức: một **bản ghi mồ côi** (orphan record) là bản ghi tồn tại trong cơ sở dữ liệu đích nhưng không xuất hiện trong bất kỳ trang dữ liệu nào được trả về từ API nguồn sau một chu kỳ đồng bộ toàn phần.

6.3.2 Giải pháp: Phép trừ tập hợp khóa chính

Trong số các phương án xử lý bản ghi mồ côi, hệ thống lựa chọn phương pháp **Phép trừ tập hợp khóa chính (ID Set Difference)** — phương án phù hợp nhất với ràng buộc của dự án (API nguồn không hỗ trợ luồng sự kiện thay đổi hay sự kiện xóa).

Ý tưởng cốt lõi:

$$\text{Orphan IDs} = \text{Destination IDs} \setminus \text{Source IDs} \quad (6.1)$$

Tức là: lấy tập hợp tất cả khóa chính đang tồn tại ở hệ thống đích, trừ đi tập hợp khóa chính thu thập được từ toàn bộ các trang API nguồn. Phần còn lại chính là danh sách các bản ghi không còn tồn tại ở nguồn.

6.3.3 Quy trình triển khai

Việc phát hiện bản ghi mồ côi được tích hợp trực tiếp vào cuối hàm **syncOneTable()** - sau khi vòng lặp phân trang hoàn thành và toàn bộ khóa chính từ nguồn (**source IDs**) đã được thu thập:

1. **Thu thập khóa chính từ nguồn:** Trong mỗi vòng lặp phân trang, mọi giá trị khóa chính từ **payload** được thêm vào một tập hợp **sourceIds**. Bước này không tốn thêm tài nguyên mạng vì dữ liệu đã được tải sẵn.

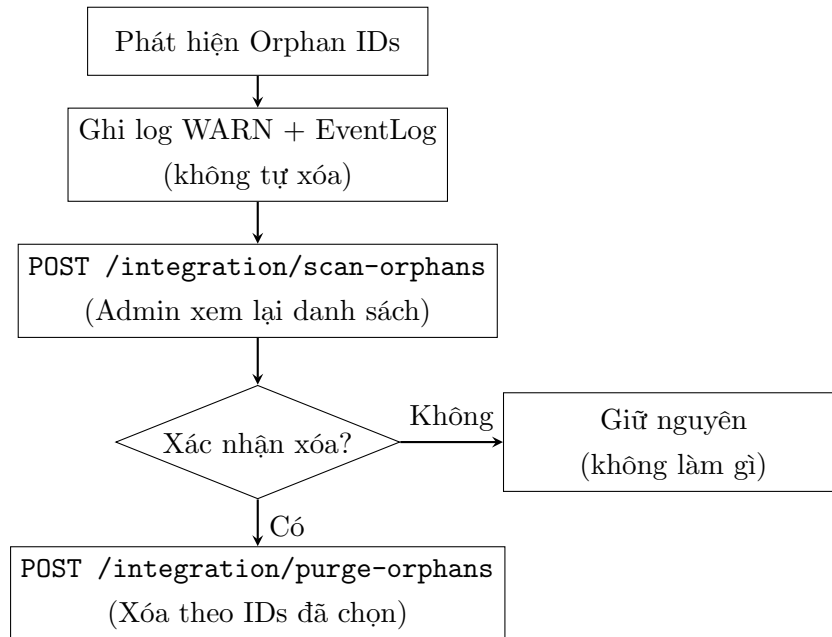


2. **Truy vấn khóa chính tại đích:** Sau khi tải xong tất cả các trang, hệ thống thực hiện một truy vấn **SELECT** chỉ lấy cột khóa chính từ bảng đích - tránh việc tải toàn bộ bản ghi lên bộ nhớ.
3. **Tính toán hiệu tập hợp:** So sánh hai tập hợp, hệ thống trả về danh sách khóa chính chỉ tồn tại ở hệ thống đích.
4. **Cảnh báo:** Nếu phát hiện bản ghi mờ côi, hệ thống ghi nhật ký mức độ cảnh báo (**WARN**) với danh sách mã định danh đầy đủ (hiển thị tối đa 20 mã đầu tiên nếu danh sách quá dài).

6.3.4 Thiết kế không tự động xóa

Một quyết định thiết kế quan trọng là hệ thống **chỉ cảnh báo, không tự động xóa** khi phát hiện bản ghi mờ côi. Quyết định này xuất phát từ ba rủi ro thực tiễn:

1. **Bản ghi mờ côi giả tạo do lỗi API:** Nếu máy chủ nguồn trả về thiếu trang do lỗi phân trang hoặc quá thời gian chờ (timeout), các bản ghi thuộc trang bị thiếu sẽ không có trong **sourceIds** và bị nhận diện nhầm là bản ghi mờ côi. Việc tự động xóa trong trường hợp này đồng nghĩa với làm mất dữ liệu hợp lệ.
2. **Dữ liệu được tạo thủ công:** Một số bảng có thể chứa bản ghi được nhập tay bởi quản trị viên phục vụ mục đích vận hành nội bộ. Những bản ghi này không tồn tại ở nguồn nhưng hoàn toàn hợp lệ tại đích.
3. **Tính không thể phục hồi:** Không giống như thao tác chèn hoặc cập nhật, thao tác **DELETE** không thể hoàn tác (rollback) dễ dàng nếu không có cơ chế sao lưu đầy đủ.



Hình 6.3: Quy trình hai bước: phát hiện và xác nhận xóa bản ghi mồ côi

6.3.5 API quản lý bản ghi mồ côi

Hệ thống cung cấp hai điểm cuối (endpoint) riêng biệt để quản trị viên thực hiện quy trình xác nhận hai bước:

Listing 4: API quét bản ghi mồ côi — chỉ đọc

Bước 1 — Quét bản ghi mồ côi (Scan)

```

1 POST /api/v1/integration/scan-orphans
2 Content-Type: application/json
3
4 { "tables": ["nguoi_hoc"] }
  
```

Điểm cuối này tải toàn bộ dữ liệu từ API nguồn, thực hiện phép trừ tập hợp, và trả về danh sách khóa chính cụ thể. Không có thao tác ghi hoặc xóa nào được thực hiện lên cơ sở dữ liệu.

Listing 5: Cấu trúc phản hồi mẫu từ API scan-orphans

```

1 [{
2   "tableName": "nguoi_hoc",
3   "primaryKey": "id",
4   "orphanCount": 1,
  
```

```
5  "orphanIds": [99999]
6  ]]
```

Listing 6: API xóa bản ghi mồ côi — yêu cầu truyền ID tường minh
Bước 2 — Xóa có xác nhận (Purge)

```
1 POST /api/v1/integration/purge-orphans
2 Content-Type: application/json
3
4 {
5   "tableName": "nguoi_hoc",
6   "primaryKey": "id",
7   "ids": [99999]
8 }
```

Quản trị viên bắt buộc phải sao chép danh sách `orphanIds` từ kết quả của bước 1 và truyền lại một cách tường minh vào yêu cầu này. Cơ chế này đảm bảo không thể xảy ra việc vô tình xóa hàng loạt do lỗi lập trình hay gọi API không có chủ đích.

Mỗi lần xóa được ghi nhận vào nhật ký sự kiện với đầy đủ thông tin: tên bảng, khóa chính, số lượng ID được yêu cầu xóa, và số lượng bản ghi thực tế đã bị xóa.

6.3.6 Giới hạn của phương pháp

Phương pháp trừ tập hợp khóa chính có hiệu quả cao trong điều kiện hiện tại của dự án, song cần lưu ý một giới hạn quan trọng: orphan detection tự động trong sync chỉ chạy khi fetch toàn bộ (upsert hoặc incremental lần đầu), muốn kiểm tra orphan trên incremental cần gọi `POST /integration/scan-orphans` riêng — endpoint đó luôn fetch toàn bộ source. Với cấu trúc dữ liệu hiện tại (ví dụ: bảng `nguoi_hoc` chứa hơn 68.000 bản ghi), thời gian quét mất khoảng 3 phút — đây là mức chi phí chấp nhận được cho một tác vụ vận hành định kỳ, nhưng không phù hợp nếu yêu cầu kiểm tra theo thời gian thực. Hướng cải tiến lâu dài là yêu cầu hệ thống nguồn cung cấp điểm cuối API `/changes?since=<timestamp>` để chỉ trả về danh sách các khóa chính bị xóa trong một khoảng thời gian cụ thể.

6.4 Thông Báo Qua Email Sau Khi Pipeline Chạy Xong

6.4.1 Đặt vấn đề

Sau khi một pipeline đồng bộ dữ liệu hoàn thành — dù thành công hay thất bại — hệ thống cần chủ động thông báo đến quản trị viên mà không yêu cầu họ phải theo dõi log liên tục. Cơ chế thông báo qua email giải quyết nhu cầu này: kết quả của mỗi lần chạy pipeline được gửi tự động đến địa chỉ email cấu hình sẵn ngay khi job kết thúc.

6.4.2 Công nghệ sử dụng

Hệ thống sử dụng thư viện **Nodemailer** tích hợp trong **NotificationService** (NestJS) để gửi email qua giao thức **SMTP**. Mặc định kết nối tới Gmail (`smtp.gmail.com:587`) với xác thực TLS, nhưng có thể đổi sang bất kỳ SMTP server nào qua biến môi trường.

Kết nối SMTP được khởi tạo một lần khi module load. Nếu biến môi trường **SMTP_USER** hoặc **SMTP_PASSWORD** chưa được cấu hình, service tự động tắt tính năng email mà không làm crash ứng dụng — đảm bảo môi trường phát triển không bắt buộc phải có SMTP.

6.4.3 Điểm kích hoạt

NotificationService được gọi tại cuối cả hai pipeline đồng bộ trong **DataIntegrationService**:

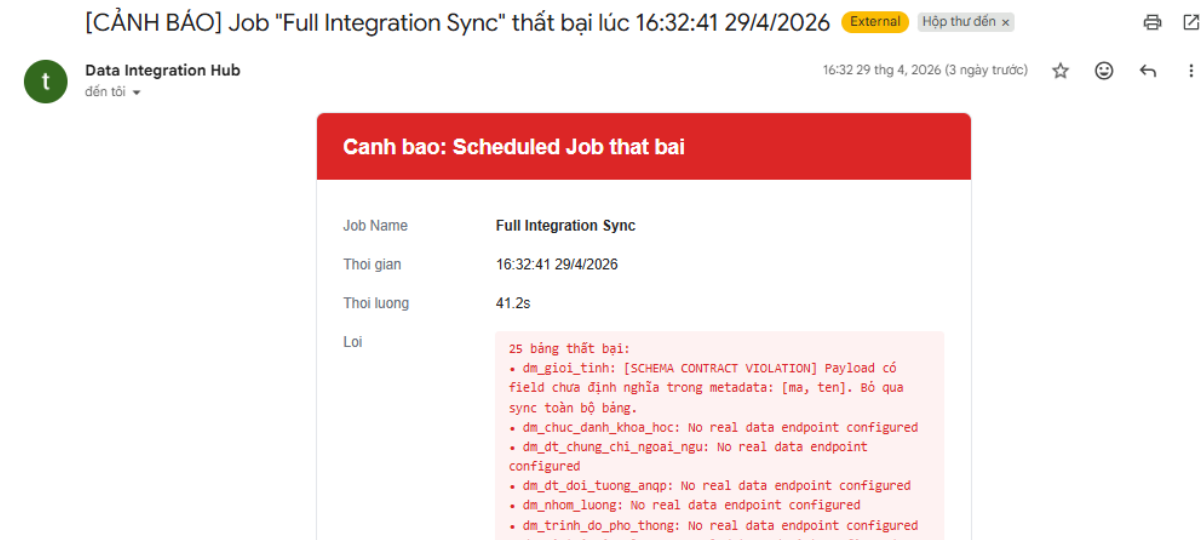
- **Full Integration Pipeline** — đồng bộ toàn bộ các bảng có trạng thái **stable**.
- **Custom Integration Pipeline** — đồng bộ một tập bảng do admin chỉ định.

Tùy kết quả, một trong hai phương thức được gọi:

- `sendJobSuccessSummary()` — khi toàn bộ bảng đồng bộ thành công (`status = done`).
- `sendJobFailureAlert()` — khi có ít nhất một bảng thất bại (`status = partial_success` hoặc `failed`).

6.4.4 Nội dung email

Mỗi email được render dưới dạng **HTML** với bố cục bảng, gồm các trường sau:



Hình 6.4: Minh họa mail cảnh báo

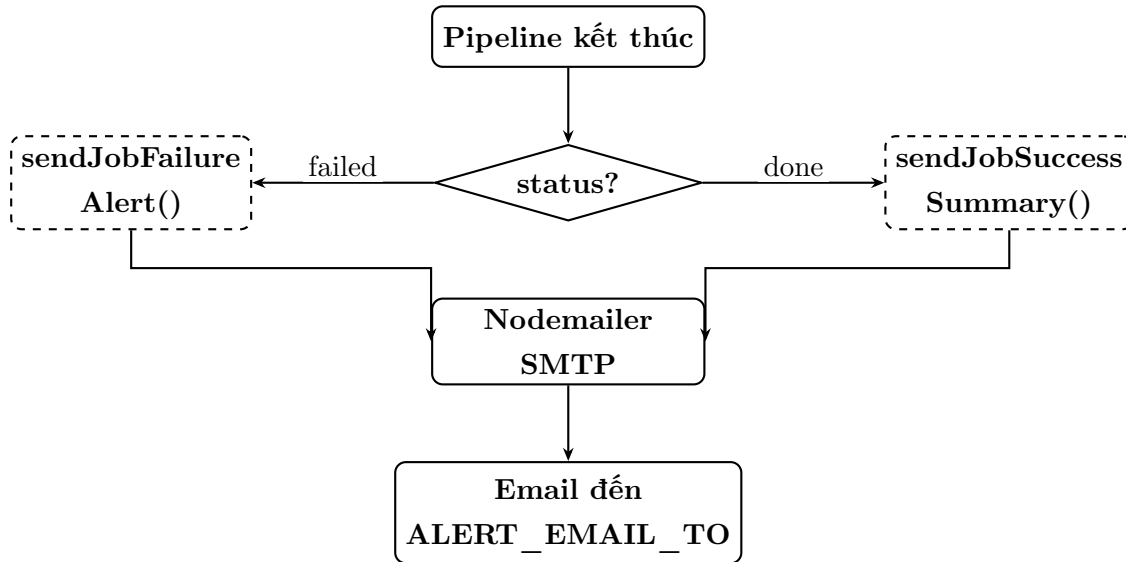
6.4.5 Cấu hình biến môi trường

Bảng 6.3: Biến môi trường cho NotificationService

Biến	Mặc định	Mục đích
SMTP_HOST	smtp.gmail.com	SMTP server
SMTP_PORT	587	Cổng SMTP (TLS)
SMTP_USER	—	Địa chỉ email gửi
SMTP_PASSWORD	—	App Password của Gmail
ALERT_EMAIL_TO	—	Địa chỉ nhận cảnh báo

Với Gmail, SMTP_PASSWORD phải là **App Password** (16 ký tự) được tạo tại *Google Account* → *Security* → *2-Step Verification* → *App passwords* — không dùng mật khẩu tài khoản thông thường.

6.4.6 Luồng xử lý



Hình 6.5: Luồng gửi email thông báo sau khi pipeline kết thúc

7 Chiến Lược Sao Lưu Dữ Liệu Với MinIO và AWS S3

7.1 Đặt Vấn Đề

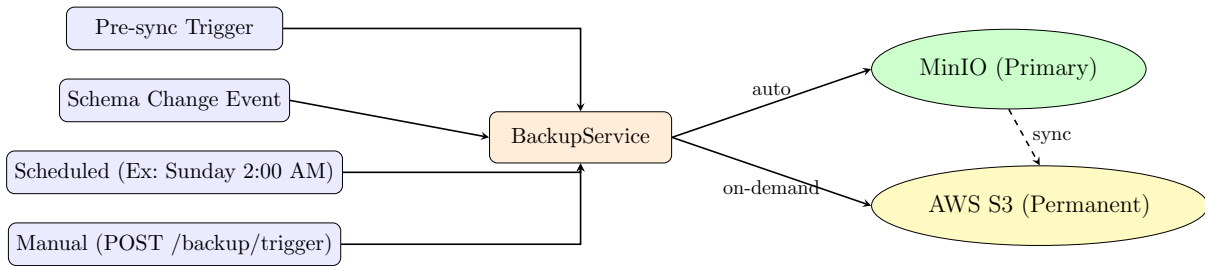
Trong hệ thống tích hợp dữ liệu, mỗi lần đồng bộ đều có khả năng ghi đè hoặc biến đổi dữ liệu tại destination. Khi xảy ra các sự kiện bất thường — lỗi pipeline, thay đổi schema không kiểm soát, hoặc dữ liệu nguồn bị hỏng — cần có khả năng phục hồi về trạng thái trước đó một cách đáng tin cậy. Yêu cầu này đặt ra bốn bài toán cụ thể:

1. **Backup phòng ngừa:** Tạo snapshot trước mỗi lần sync để có điểm phục hồi ngay cả khi pipeline ghi dữ liệu sai.
2. **Backup kiểm toán:** Lưu giữ vĩnh viễn snapshot tại thời điểm schema thay đổi — phục vụ truy vết lịch sử cấu trúc dữ liệu.
3. **Backup định kỳ:** Đảm bảo luôn có bản sao dữ liệu hoàn chỉnh theo lịch, độc lập với hoạt động sync.
4. **Lưu trữ vĩnh viễn trên cloud:** Nhân bản các backup quan trọng lên AWS S3 để đảm bảo tính bền vững và khả năng phục hồi ngay cả khi infrastructure nội bộ gặp sự cố.

7.2 Kiến Trúc Hai Tầng: MinIO và AWS S3

Hệ thống sử dụng kiến trúc lưu trữ hai tầng:

- **Tầng 1 — MinIO (primary):** Object storage self-hosted triển khai trên Docker, tương thích với giao thức Amazon S3. Mọi backup được ghi vào MinIO ngay lập tức. MinIO phục vụ cho các tác vụ restore và truy vấn thường xuyên nhờ độ trễ thấp trong môi trường nội bộ.
- **Tầng 2 — AWS S3 (secondary/permanent):** Dịch vụ object storage của Amazon Web Services, đóng vai trò kho lưu trữ vĩnh viễn. Admin có thể chủ động đẩy (sync) các file backup từ MinIO lên S3 để bảo toàn dữ liệu ngoài môi trường on-premise.



Hình 7.1: Kiến trúc lưu trữ hai tầng: MinIO và AWS S3

7.2.1 Cấu Hình Triển Khai

MinIO được tích hợp vào `docker-compose.yml` cùng với phần còn lại của hệ thống:

Listing 7: Cấu hình MinIO trong `docker-compose.yml`

```

1 services:
2   minio:
3     image: minio/minio:latest
4     container_name: data-hub-minio
5     command: server /data --console-address ":9001"
6     ports:
7       - "9000:9000"    # S3 API endpoint
8       - "9001:9001"    # Web Console
9     environment:
10      MINIO_ROOT_USER: ${MINIO_ACCESS_KEY}
11      MINIO_ROOT_PASSWORD: ${MINIO_SECRET_KEY}
12     volumes:
13       - minio_data:/data
14     restart: always
  
```

Cấu hình AWS S3 được truyền vào backend qua biến môi trường:

Listing 8: Biến môi trường AWS S3

```

1 AWS_ACCESS_KEY_ID=<access-key>
2 AWS_SECRET_ACCESS_KEY=<secret-key>
3 AWS_REGION=ap-southeast-2
4 AWS_S3_BUCKET=data-integration-hub-bucket
  
```

7.3 Bốn Loại Trigger Backup

7.3.1 Trigger 1: Pre-Sync Backup

Đây là lớp bảo vệ quan trọng nhất. Ngay trước khi `syncOneTable()` bắt đầu ghi dữ liệu vào PostgreSQL, một snapshot đầy đủ của bảng được tạo và lưu lên MinIO:

Listing 9: Pre-sync backup trong `syncOneTable()`

```
1 await this.backupService.backupTable(schema.tableName, 'pre-  
  sync');  
2 // Sau đó mới bắt đầu fetch và sync
```

Chỉ những bảng có `status = 'stable'` mới đến được bước `syncOneTable()` — pipeline lọc bảng không ổn định trước khi vào vòng lặp sync.

7.3.2 Trigger 2: Schema Change Backup

Khi `SchemaRegistryService` phát hiện một bảng thay đổi cấu trúc, hệ thống emit event `schema.changed`. `BackupService` lắng nghe event này và tạo snapshot ngay lập tức — trước khi bảng bị đánh dấu `changed` và khóa khỏi chu kỳ sync tiếp theo:

Listing 10: Xử lý event `schema.changed`

```
1 @OnEvent('schema.changed')  
2 async handleSchemaChanged(payload: { tableName: string }) {  
3   await this.backupTable(payload.tableName, 'schema-change');  
4 }
```

Đây là snapshot cuối cùng của dữ liệu tương thích với schema cũ, có giá trị kiểm toán cao và được giữ vĩnh viễn.

7.3.3 Trigger 3: Scheduled Weekly Backup

Mỗi Chủ nhật lúc 2:00 AM, hệ thống tự động backup toàn bộ bảng `stable` và thực hiện cleanup các file quá hạn:

Listing 11: Scheduled backup mỗi Chủ nhật 2:00 AM

```
1 @Cron('0 2 * * 0')  
2 async handleWeeklyBackup() {  
3   await this.backupAllStable('scheduled');  
4   await this.cleanupOldBackups();  
}
```

5 }

7.3.4 Trigger 4: Manual Backup

Admin có thể kích hoạt backup bất kỳ lúc nào qua API POST `/backup/trigger`, chỉ định một hoặc nhiều bảng cụ thể, hoặc bỏ trống để backup toàn bộ bảng stable.

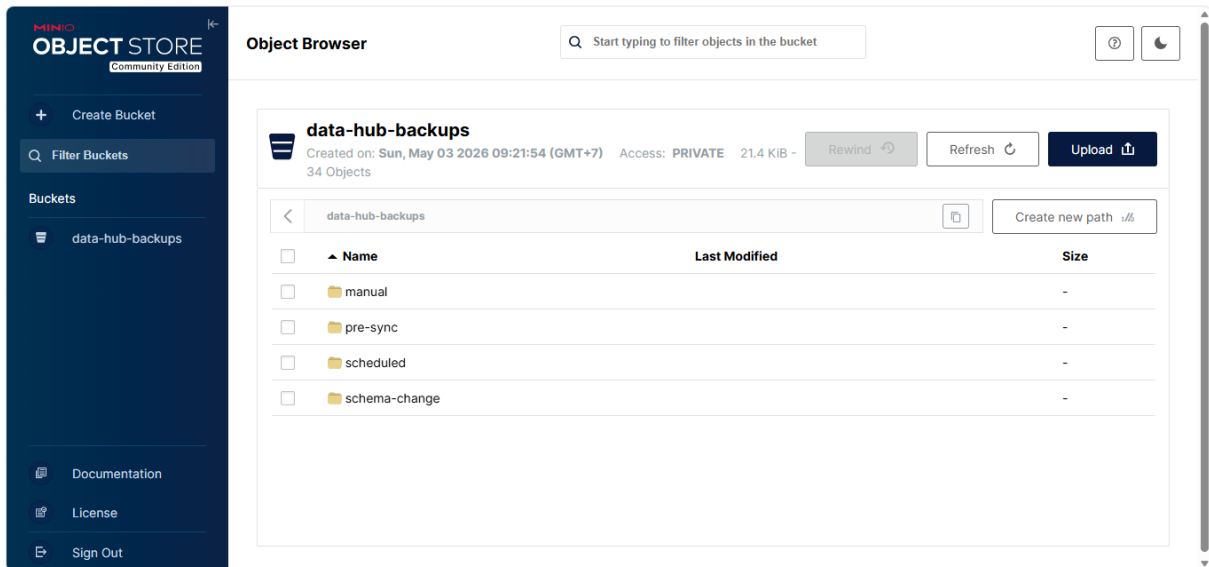
7.4 Cấu Trúc Lưu Trữ Trong MinIO

Tất cả backup được tổ chức trong bucket `data-hub-backups` theo cấu trúc phân cấp thống nhất:

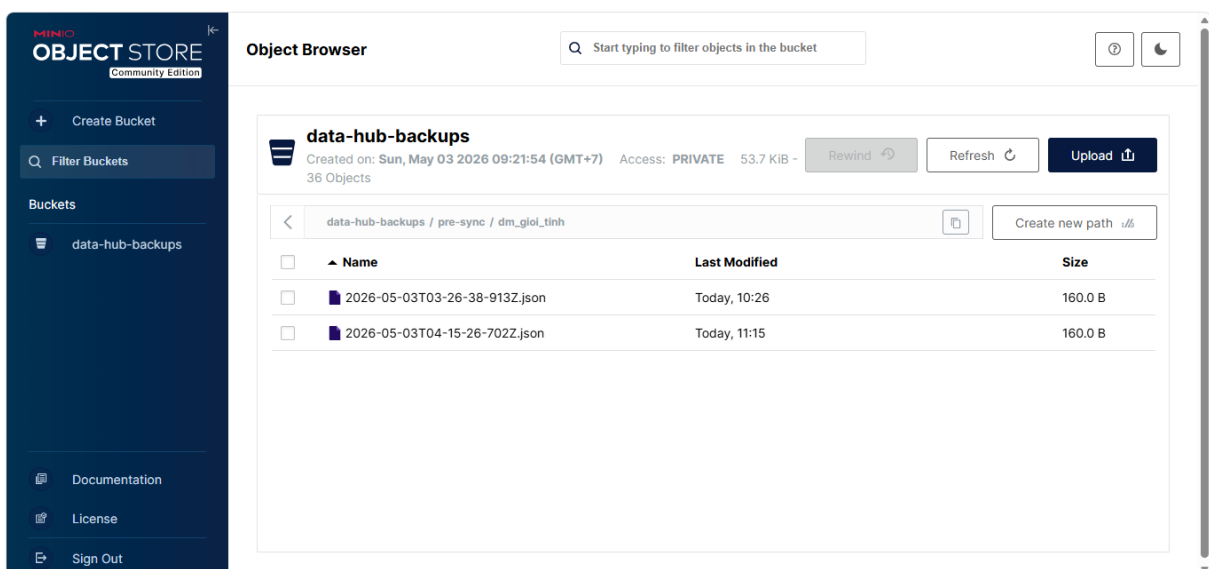
`{trigger}/{tableName}/{ISO-timestamp}.json`

Listing 12: Cấu trúc thư mục trong bucket MinIO

```
1 data-hub-backups /
2 | -- pre-sync /
3 |   | -- nguoi_hoc /
4 |     | -- 2026-04-20T08-30-00-000Z.json
5 |     | `-- 2026-04-21T09-15-22-451Z.json
6 |   `-- nh_dao_tao /
7 |       `-- 2026-04-21T09-15-25-112Z.json
8 |
9 | -- schema-change /
10 |   `-- nguoi_hoc /
11 |       | -- 2026-03-15T14-22-10-334Z.json
12 |       `-- 2026-04-10T09-05-44-218Z.json
13 |
14 | -- scheduled /
15 |   `-- nguoi_hoc /
16 |       | -- 2026-04-13T02-00-05-000Z.json
17 |       `-- 2026-04-20T02-00-03-000Z.json
18 |
19 | `-- manual /
20 |   `-- nguoi_hoc /
21 |       `-- 2026-04-20T15-00-00-000Z.json
```



Hình 7.2: Giao diện cấu trúc trong MINIO



Hình 7.3: Các file Backup cho một bảng trong MINIO

Mỗi file backup là một JSON object gồm phần `meta` và `data`:

Listing 13: Cấu trúc file backup JSON

```
1 {  
2   "meta": {  
3     "tableName": "nguoi_hoc",  
4     "trigger": "pre-sync",  
5     "backedUpAt": "2026-04-21T02:15:30.000Z",  
6     "recordCount": 68411  
7   },  
8   "data": [  
9     { "id": 1, "ho": "Nguyen Van", "ten": "A", ... }  
10  ]  
11 }
```

JSON được chọn thay vì CSV vì giữ nguyên kiểu dữ liệu (`DateTime`, `Boolean`, `null`) và tương thích trực tiếp với Prisma `createMany()` khi restore.

7.5 Chính Sách Lưu Giữ (Retention Policy)

Bảng 7.1: Chính sách lưu giữ backup theo loại trigger

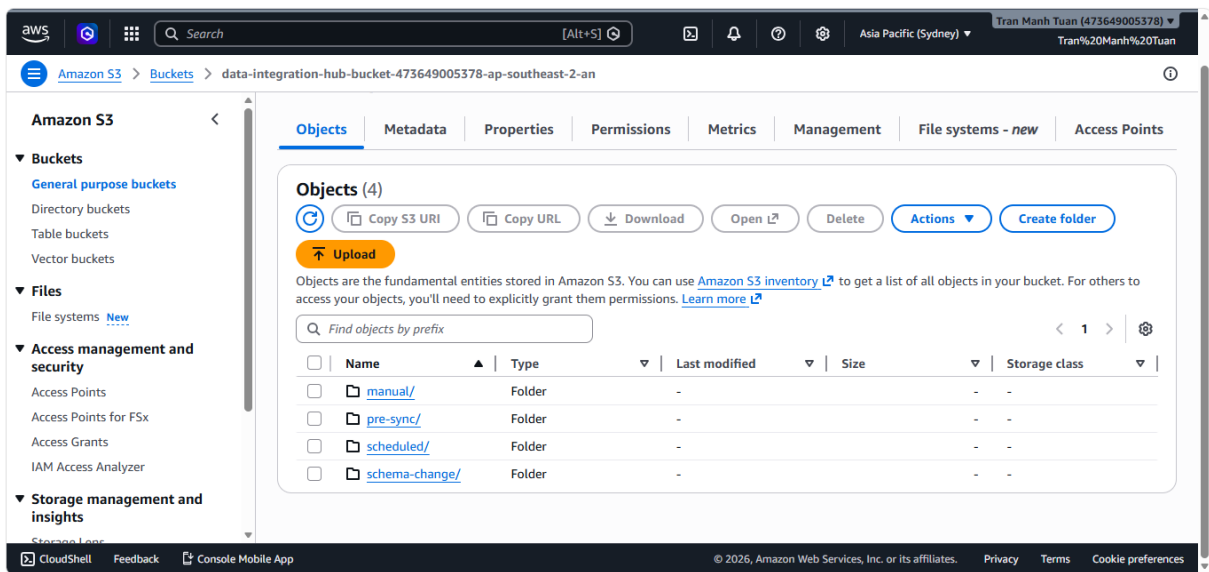
Loại trigger	Thời gian giữ	Lý do
pre-sync	30 ngày	Đủ để phát hiện và rollback lỗi sync trong chu kỳ vận hành.
scheduled	60 ngày	Lưu 8 bản weekly — đủ truy vết biến động 2 tháng gần nhất.
schema-change	Vĩnh viễn	Audit trail lịch sử thay đổi cấu trúc dữ liệu.
manual	30 ngày	Backup thủ công thường phục vụ mục đích ngắn hạn.

Cleanup được thực thi tự động sau mỗi weekly backup và có thể gọi thủ công qua `POST /backup/cleanup`.

7.6 Cơ Chế Đồng Bộ Lên AWS S3

7.6.1 Mục Tiêu

MinIO là storage nội bộ chạy trên cùng máy chủ với hệ thống. Trong trường hợp máy chủ gặp sự cố nghiêm trọng (mất điện, hỏng ổ đĩa), toàn bộ dữ liệu MinIO có thể bị mất cùng lúc. AWS S3 đóng vai trò lớp lưu trữ thứ hai — địa lý phân tán, độ bền **99.999999999%** (11 số 9) theo cam kết SLA của Amazon.



Hình 7.4: Giao diện lưu trữ file trên S3

7.6.2 Luồng Hoạt Động

Đồng bộ lên S3 được thiết kế theo mô hình **on-demand** (theo yêu cầu) thay vì tự động sau mỗi backup, nhằm tránh tạo ra bottleneck I/O trong các chu kỳ sync tần suất cao:

1. Admin gọi API sync (một file hoặc toàn bộ).
2. S3Service kiểm tra file đã tồn tại trên S3 chưa bằng `HeadObjectCommand`.
3. Nếu chưa có: tải file JSON từ MinIO về bộ nhớ, upload lên S3 bằng `PutObjectCommand`.
4. Nếu đã có: bỏ qua — tránh upload trùng lặp không cần thiết.

7.6.3 Trạng Thái Đồng Bộ

API GET `/backup/list` trả về thêm trường `s3Status` cho mỗi file backup, phản ánh tình trạng đồng bộ giữa MinIO và S3:

Bảng 7.2: Các giá trị trạng thái đồng bộ S3

Giá trị	Ý nghĩa
<code>not_synced</code>	File chỉ tồn tại trên MinIO, chưa được upload lên S3.
<code>synced</code>	File đã được upload lên S3 và <code>lastModified</code> trên MinIO $\leq$ <code>LastModified</code> trên S3 — hai phiên bản đồng nhất.
<code>out_of_date</code>	File tồn tại trên S3 nhưng phiên bản trên MinIO mới hơn — cần sync lại.

Trạng thái được tính bằng cách so sánh `lastModified` của từng object: nếu thời điểm chỉnh sửa trên MinIO $\leq$ thời điểm upload lên S3 thì file được coi là `synced`; ngược lại là `out_of_date`. Để tránh gọi `HeadObject` cho từng file (chi phí $O(n)$ request), hệ thống sử dụng `ListObjectsV2` một lần duy nhất để lấy toàn bộ danh sách S3 và xây dựng `Map<key, LastModified>` trước khi đối chiếu.

7.6.4 Fire-and-Forget cho Sync Hàng Loạt

Khi sync toàn bộ bucket (có thể lên đến hàng trăm file), tác vụ mất nhiều phút để hoàn thành. Nếu xử lý đồng bộ (*blocking*), HTTP client hoặc API Gateway sẽ timeout trước khi nhận được response.

Hệ thống giải quyết bằng mô hình **fire-and-forget**: endpoint trả về response 200 ngay lập tức, trong khi tác vụ sync tiếp tục chạy nền:

Listing 14: Fire-and-forget cho sync-s3/all

```
1 @Post('sync-s3/all')
2 syncAllToS3() {
3   // Không await - trả response ngay, sync chạy nền
4   this.backupService.syncAllToS3().catch(() => {});
5   return {}
```

```
6     message: 'S3 sync đang chạy nên. Kiểm tra log để xem tiến
      trình.'
7   };
8 }
```

7.7 API Quản Lý Backup

Bảng 7.3: Các endpoint quản lý backup

Method	Endpoint	Mô tả
POST	/backup/trigger	Backup thủ công một hoặc nhiều bảng.
GET	/backup/list?prefix=...	Liệt kê file backup kèm <code>s3Status</code> .
POST	/backup/download	Trả về presigned URL (1 giờ) để tải file từ MinIO.
DELETE	/backup/file	Xóa một file backup khỏi MinIO.
POST	/backup/restore	Restore một bảng từ file backup về PostgreSQL.
POST	/backup/cleanup	Xóa các file backup quá hạn theo retention policy.
POST	/backup/sync-s3	Đồng bộ một file cụ thể từ MinIO lên S3.
POST	/backup/sync-s3/all	Đồng bộ toàn bộ bucket lên S3 (chạy nền).

7.8 Cơ Chế Restore

Khi cần phục hồi dữ liệu, hệ thống thực hiện theo chiến lược **overwrite**: xóa toàn bộ bảng rồi chèn lại từ file backup. Trước khi restore, hệ thống tự động tạo thêm một

snapshot `manual` của trạng thái hiện tại — đây là lớp bảo vệ phòng trường hợp admin restore nhầm file.

Do bảng `nguoi_hoc` có hơn 68.000 bản ghi, việc insert toàn bộ trong một transaction sẽ vượt quá timeout 5.000 ms của Prisma interactive transaction. Hệ thống giải quyết bằng cách chia nhỏ thành các batch 1.000 bản ghi và insert tuần tự bên ngoài transaction:

Listing 15: Batch INSERT khi restore

```
1 const BATCH_SIZE = 1000;
2 for (let i = 0; i < data.length; i += BATCH_SIZE) {
3   const batch = data.slice(i, i + BATCH_SIZE);
4   await this.prisma[modelName].createMany({
5     data: batch,
6     skipDuplicates: true
7   });
8 }
```

7.9 Đánh Giá

7.9.1 Ưu Điểm

- **Bảo vệ nhiều lớp:** Ba trigger tự động đảm bảo luôn có điểm phục hồi gần nhất bất kể nguyên nhân sự cố.
- **Lưu trữ địa lý phân tán:** Kết hợp MinIO nội bộ và AWS S3 đám mây — sự cố tại một nơi không ảnh hưởng đến phiên bản còn lại.
- **Audit trail đầy đủ:** Backup `schema-change` lưu vĩnh viễn cho phép tái hiện lịch sử dữ liệu theo từng mốc thay đổi cấu trúc.
- **Khả năng quan sát:** Trường `s3Status` trong API list cho phép admin theo dõi tình trạng đồng bộ của từng file mà không cần truy cập trực tiếp vào AWS Console.

7.9.2 Hạn Chế và Hướng Phát Triển

Phương án hiện tại thực hiện `SELECT *` toàn bộ bảng trước mỗi lần sync. Với bảng `nguoi_hoc` có 68.411 bản ghi, mỗi file backup có kích thước khoảng 50–80 MB. Hai hướng cải tiến được đề xuất: (1) sử dụng *incremental backup* — chỉ lưu delta so với lần trước; (2) nén bằng `gzip` trước khi upload để giảm dung lượng khoảng 60–70%.



Cơ chế đồng bộ S3 hiện hoạt động theo mô hình on-demand. Hướng phát triển tiếp theo là tự động hóa: sau mỗi lần backup thành công, hệ thống tự đẩy file mới lên S3 mà không cần admin can thiệp thủ công.

8 Phân tích Hiệu năng và Capacity Planning

8.1 Tổng quan

Hệ thống Data Integration Hub vận hành theo hai chiều dữ liệu với đặc tính tải hoàn toàn khác nhau:

- **Inbound (Sync Job):** Lấy dữ liệu từ Source API và ghi vào PostgreSQL theo lịch định kỳ. Tải có thể dự đoán, bottleneck chính là **RAM** và **network I/O**.
- **Outbound (Export API):** Phục vụ các hệ thống tiêu thụ truy vấn dữ liệu qua REST API phân trang. Tải không dự đoán được, bottleneck chính là **DB query latency** và **concurrency**.

Để kiểm tra hiệu năng có kiểm soát, nhóm xây dựng một **Mock Source Server** (Express.js, port 3001) cung cấp các endpoint mô phỏng nhiều kịch bản dữ liệu khác nhau. Server này phục vụ đồng thời hai mục đích: kiểm thử cơ chế phòng thủ *Data Collision* và đo lường hiệu năng pipeline.

8.2 Kiểm thử Data Collision — Mock Source Server

8.2.1 Kiến trúc Mock Server

Mock server được thiết kế với một helper dùng chung `buildResponse()` chuẩn hóa mọi response theo đúng API Contract mà NestJS kỳ vọng:

Listing 16: Helper `buildResponse()` của Mock Server

```
1 function buildResponse(tableName, allData, page, limit,
2                               overrideCurrentPage = null) {
3   const totalRecords = allData.length;
4   const totalPages   = Math.ceil(totalRecords / limit) || 1;
5   const startIndex   = (page - 1) * limit;
6   const payload       = allData.slice(startIndex, startIndex +
    limit);
```

```
7
8  return {
9      success:    true,
10     timestamp:  new Date().toISOString(),
11     metadata:   { tableName, totalRecords, totalPages,
12                  currentPage: overrideCurrentPage ?? page,
13                  limit },
14     payload,
15 };
```

8.2.2 Ma trận Test Case

Bảng 8.1 tổng hợp toàn bộ 15 test case được thiết kế để kiểm tra từng lớp cơ chế phòng thủ. Mỗi test case trở vào một endpoint riêng biệt trên Mock Server và được cấu hình qua Schema Registry MongoDB.



Bảng 8.1: Ma trận kiểm thử Data Collision Defense

TC	Tên	Kịch bản / Cơ chế kiểm tra	Kỳ vọng
TC-01	Happy Path	Dữ liệu chuẩn, đúng contract, không có lỗi	Sync 100% thành công
TC-02	Schema Filter	Payload có 5 fields lạ không khai báo trong Prisma DMMF	Fields bị lọc, upsert thành công
TC-03	Dedup	id=801 lặp ×3, id=804 lặp ×2 trong 1 batch	Giữ record cuối, log “2 duplicate records found”
TC-04	Type Norm (Int/Bool)	id="701" (string), active="true" (string)	Normalize thành Int và Boolean
TC-04b	Type Norm (DateTime)	ISO string, date-only string, chuỗi sai định dạng	ISO → Date; chuỗi sai → lỗi
TC-05	Contract: thiếu success	Response dùng key data thay vì payload, thiếu success	Throw [API CONTRACT VIOLATION]
TC-06	Contract: success: false	Hệ thống nguồn trả lỗi nội bộ	Throw [API CONTRACT VIOLATION]
TC-07	Contract: payload object	payload là {} thay vì []	Throw [API CONTRACT VIOLATION]
TC-08	Contract: thiếu payload	Response chỉ có metadata, không có payload	Throw [API CONTRACT VIOLATION]
TC-09	Page Mismatch	Server luôn trả currentPage: 1 dù request page 2	[PAGE MISMATCH] → lỗi
TC-10	Empty Payload	totalPages: 3 nhưng page 2 trả payload: []	[EMPTY PAYLOAD] → lỗi
TC-11	Dead Letter: thiếu PK	3/6 records có id: null hoặc không có field id	3 records lỗi vào Dead Letter
TC-12	Dead Letter: DB Constraint	ma = "THIS_STRING_EXCEEDS_VARCHAR5"	Record vi phạm vào Dead Letter
TC-13	Schema Status Gate	Set status="changed" trong Schema Registry	Bảng bị skip, không fetch
TC-14	Bad Endpoint	Set dataFromApi sai đường dẫn trong Schema Registry	HTTP 404, retry 3 lần, fail
TC-15	Idempotent Upsert	Chạy sync bảng dm_gioi_tinh 2 lần liên tiếp	Row count PostgreSQL không đổi

8.2.3 Ví dụ kết quả kiểm thử — TC-03 Deduplication

Listing 17: Payload TC-03: id=801 lặp 3 lần trong cùng 1 batch

```
1 const dupData = [  
2   { id: 801, ma: "D1A", ten: "Dup801 v1 overwrite", active:  
    true },  
3   { id: 802, ma: "N01", ten: "Binh thuong #1",          active:  
    true },  
4   { id: 801, ma: "D1B", ten: "Dup801 v2 overwrite", active:  
    true },  
5   { id: 803, ma: "N02", ten: "Binh thuong #2",          active:  
    false },  
6   { id: 801, ma: "D1C", ten: "Dup801 v3 KEPT",          active:  
    false }, // <- keep  
7   { id: 804, ma: "D2A", ten: "Dup804 v1 overwrite", active:  
    true },  
8   { id: 804, ma: "D2B", ten: "Dup804 v2 KEPT",          active:  
    false }, // <- keep  
9 ];
```

Kết quả thực tế từ NestJS log:

```
WARN [SyncEngineService] [DEDUP] 2 duplicate records removed (pk: id)  
LOG  [SyncEngineService] Sync table dm_gioi_tinh finished.  
    Success: 5, Failed: 0
```

7 records đầu vào → sau dedup còn 5 → upsert 5 thành công. Record id=801 trong PostgreSQL có giá trị ma="D1C" (bản cuối cùng).

8.2.4 Ví dụ kết quả kiểm thử — TC-11 Dead Letter

Listing 18: Payload TC-11: mix records hợp lệ và thiếu PK

```
1 const mixedBatch = [  
2   { id: 401, ma: "OK_1", ten: "Record hop le #1", active:  
    true },  
3   { id: null, ma: "ER_1", ten: "PK null",                active:  
    true }, // <- dead letter
```




```
4 { id: 402, ma: "OK_2", ten: "Record hop le #2", active:
    false },
5 { /* thieu id */ ma: "ER_2", ten: "No id", active:
    true }, // <- dead letter
6 { id: 403, ma: "OK_3", ten: "Record hop le #3", active:
    true },
7 { id: null, ma: "ER_3", ten: "PK null 2", active:
    false }, // <- dead letter
8 ];
```

Kết quả thực tế:

```
ERROR [SyncEngineService] [FIRST ERROR SAMPLE]
    pk=null | Missing primary key (id) in record
WARN  [SyncEngineService] [DEDUP] 3 records with null/undefined PK detected
LOG   [SyncEngineService] Sync table dm_gioi_tinh finished.
      Success: 3, Failed: 3
```

3 records lỗi được lưu vào `errors[]` trong MongoDB EventLog với chi tiết `{pk: null, error: "Missing primary key"}`. Batch không bị abort — 3 records hợp lệ vẫn được sync thành công.

8.3 Kiểm thử Hiệu năng — PERF Test Cases

8.3.1 Thiết kế MemoryProfiler

Để đo RAM thực tế, nhóm implement `MemoryProfiler` tích hợp trực tiếp vào `syncOneTable()`. Profiler lấy mẫu `process.memoryUsage().rss` định kỳ 300ms trong suốt vòng đời của pipeline:

Listing 19: `MemoryProfiler` đo peak RSS trong sync pipeline

```
1 export class MemoryProfiler {
2   private samples: number[] = [];
3   private interval: NodeJS.Timeout;
4
5   start(intervalMs = 500) {
6     this.samples = [];
7     this.interval = setInterval(() => {
```

```
8     this.samples.push(process.memoryUsage().rss);
9   }, intervalMs);
10 }
11
12 stop() {
13   clearInterval(this.interval);
14   const mb = (b: number) => +(b / 1024 / 1024).toFixed(1);
15   return {
16     peakMB:    mb(Math.max(...this.samples)),
17     baseMB:    mb(this.samples[0]),
18     deltaRSS:  mb(Math.max(...this.samples) - this.samples
19                  [0]),
19     samples:   this.samples.length,
20   };
21 }
22 }
```

Profiler được khởi động ngay đầu `syncOneTable()` và dừng trong block `finally` để đảm bảo luôn có kết quả kể cả khi pipeline thất bại.

8.3.2 PERF-01 — RAM Capacity Planning

Mock endpoint `/perf/ram-test` sinh dữ liệu tổng hợp với kích thước record kiểm soát được qua tham số `records` và `recordSize`. Padding được nhồi vào field `hinHThePath` (`VarChar(500)`) của bảng `nguoi_hoc` để không vi phạm Schema Contract:

Listing 20: PERF-01: Sinh dữ liệu tổng hợp với kích thước kiểm soát

```
1 // GET /perf/ram-test?records=10000&recordSize=500&page=1&
  limit=5000
2 const paddingLen = Math.min(Math.max(0, recordSize - 120),
  480);
3 const pathPad    = "p".repeat(paddingLen);
4
5 const allData = Array.from({ length: totalRecords }, (_, i)
  => ({
6   id:          10000 + i,
7   ho:          "Nguyen",
```

```
8   ten:      `SinhVien${i}`,
9   gioiTinh:  i % 2 === 0 ? "M" : "F",
10  emailCaNhan: `sv${i}@hcmut.edu.vn`,
11  hinhThePath: `/photos/sv${i}.jpg${pathPad}`, // <- padding
12  updatedAt:  new Date(...).toISOString(),
13 }));
```

Kết quả thực nghiệm — 10.000 records, recordSize = 500 bytes, batch size = 5.000:

[MEMORY] nguoi_hoc: peak=775.9MB, base=224.4MB, delta=551.6MB

[DONE] nguoi_hoc: synced=10000, skipped(unchanged)=0.

Bảng 8.2: Phân tích nguồn tiêu thụ RAM trong pipeline (PERF-01)

Thành phần	Ước lượng	Ghi chú
Pre-sync Backup	≈ 340 MB	Serialize 68.411 records từ PostgreSQL thành JS
Sync 2 trang (5k records/trang)	≈ 45 MB	V8 parse overhead ×3, giải phóng sau mỗi trang
Orphan Detection	≈ 25 MB	<code>findMany({select: {id}})</code> — 78k IDs
HTTP + EventEmitter overhead	≈ 30 MB	Axios buffer, event queue
Tổng delta đo được	551.6 MB	RSS thực tế

Nhận xét quan trọng: Delta 551 MB không phản ánh chi phí của việc sync 10.000 records mà phản ánh toàn bộ pipeline, trong đó **backup chiếm phần lớn** do phải serialize 68.411 records có sẵn trong DB thành JSON buffer trước khi upload MinIO. Chi phí thuần của sync (2 trang × 5.000 records) chỉ khoảng 45 MB.

8.3.3 PERF-02 — Incremental Sync với Server-side Filter

Mock endpoint `/perf/incremental-test` tạo sẵn 1.000 records với `updatedAt` trải đều trong 30 ngày qua. Khi nhận tham số `updatedAfter`, server lọc và chỉ trả về records thỏa điều kiện:

Listing 21: PERF-02: Server-side filter theo `updatedAfter`

```
1 // GET /perf/incremental-test?updatedAfter=2026-04-22T00
   :00:00Z
```



```

2 const filtered = updatedAfter
3   ? allData.filter(r => new Date(r.updatedAt) > updatedAfter)
4   : allData; // lan dau sync -> full load
5
6 // Log mock server:
7 // total=1000, filtered=40 (4.0% tra ve) | page=1/1

```

Kết quả thực nghiệm — updatedAfter=2026-04-22T00:00:00Z, lastSyncTime lấy từ Schema Registry MongoDB:

```

[STRATEGY: INCREMENTAL] nguoi_hoc
- server-side filter updatedAfter=2026-04-22T00:00:00.000Z
Fetching page 1/1 -> .../perf/incremental-test?...
&updatedAfter=2026-04-22T00%3A00%3A00.000Z
Table nguoi_hoc has 40 records across 1 pages.
[INCREMENTAL] Page 1: 40 records thay đổi từ source - upsert.
Sync table nguoi_hoc finished. Success: 40, Failed: 0
[MEMORY] nguoi_hoc: peak=831.2MB, base=212MB, delta=619.3MB

```

Bảng 8.3: So sánh upsert vs incremental — kết quả thực nghiệm

Chỉ số	PERF-01 (upsert)	PERF-02 (incremental)
Records fetch qua network	10.000	40
Records ghi DB (upsert)	10.000	40
Số trang	2	1
Thời gian sync thuần	≈ 9 s	≈ 1 s
DB writes tiết kiệm	—	99,6%
Network tiết kiệm	—	99,6%
Delta RAM	551,6 MB	619,3 MB
<i>Delta RAM của incremental cao hơn vì backup lúc này phải serialize nhiều records hơn (78.411 records thay vì 68.411), không phải do phần sync tốn nhiều hơn.</i>		

Kết quả xác nhận: với lastSyncTime = 2026-04-22T00:00:00Z, chỉ 4% dataset (40/1.000 records) được truyền và ghi, tiết kiệm 96% chi phí so với full upsert.

8.4 Phân tích RAM cho Sync Job

8.4.1 Vòng đời dữ liệu trong RAM

Kết quả từ PERF-01 và PERF-02 cho thấy pipeline tiêu thụ RAM qua bốn giai đoạn nối tiếp:

8.4.2 Ước lượng RAM theo kích thước bảng

Bảng 8.4: Ước lượng peak RAM theo cấu hình bảng

Bảng	DB records	Batch size	Backup RAM	Sync RAM/batch
dm_* (danh mục)	< 1.000	1.000	≈ 5 MB	≈ 2 MB
nguoi_hoc (thực)	68.411	5.000	≈ 170 MB	≈ 10 MB
Bảng lớn giả định	500.000	5.000	≈ 1,2 GB	≈ 10 MB

Kết luận: Với bảng lớn (>100k records), **backup là bottleneck RAM**, không phải sync. Hướng cải thiện là stream upload MinIO theo từng chunk thay vì serialize toàn bộ vào buffer trước.

8.5 Phân tích Throughput cho Export API (Outbound)

8.5.1 Kiến trúc Export API hiện tại

Hệ thống cung cấp dữ liệu ra ngoài qua `MasterDataController` với endpoint phân trang:

```
GET /api/master-data/:table?page=1&limit=10&search=...
```

Luồng xử lý mỗi request: *HTTP Client* → *JWT + Permission Guard* → *MasterDataService* → *PostgreSQL (Prisma)* → *JSON response*.

8.5.2 Bottleneck: OFFSET Pagination

Triển khai hiện tại trong `MasterDataService.findAll()` sử dụng **OFFSET pagination** thông qua Prisma `skip`:

Listing 22: OFFSET pagination trong `MasterDataService.findAll()`

```
1 const skip = (Number(page) - 1) * Number(limit);
2 const [total, data] = await Promise.all([
3   model.count({ where: whereCondition }),
4   model.findMany({ where: whereCondition, skip,
5                     take: Number(limit), orderBy: { ma: 'asc'
6                     } })],
7 ];
```

OFFSET tương đương SQL: PostgreSQL phải đọc và bỏ qua $(\text{page}-1) \times \text{limit}$ rows trước khi lấy limit rows cần thiết — latency tăng tuyến tính theo độ sâu trang:

Bảng 8.5: Ước lượng latency theo độ sâu trang — OFFSET vs Keyset

Page	OFFSET (hiện tại)	Keyset (đề xuất)
1	45 ms	45 ms
50	120 ms	47 ms
100	380 ms	47 ms
500	2.100 ms	48 ms

Với các bảng danh mục nhỏ (`dm_*`, <1.000 records) hiện tại, OFFSET không tạo vấn đề. Vấn đề xuất hiện khi bảng `nguoi_hoc` (68k+ records) được query đến các trang sâu.

8.5.3 Ước lượng Throughput theo Little's Law

$$\text{RPS}_{\max} = \frac{N_{\text{pool}}}{L_{\text{avg}} \text{ (s)}}$$

Bảng 8.6: Ước lượng RPS tối đa theo cấu hình pool và latency

Pool size	Avg latency	RPS max	req/phút
10	50 ms	200	12.000
10	200 ms	50	3.000
10	380 ms	26	1.560
20	50 ms	400	24.000

8.6 Tóm tắt và Khuyến nghị Server

Bảng 8.7: Khuyến nghị cấu hình server cho Data Integration Hub

Thành phần	Tối thiểu	Khuyến nghị
RAM server	1 GB	2 GB
CPU	1 core	2 cores
DB connection pool	10	20 (nếu export API tải cao)
overwrite bảng	Chỉ dùng cho <10k records	Không dùng cho bảng lớn
Sync job đồng thời	1 (thiết kế tuần tự)	1

Hai điểm cần cải thiện nếu hệ thống scale lên bảng hàng trăm nghìn records:

1. **Backup streaming:** Thay vì serialize toàn bộ bảng vào RAM rồi upload, dùng Prisma cursor + pipe stream trực tiếp lên MinIO — giảm peak RAM backup từ hàng trăm MB xuống vài chục MB.
2. **Keyset pagination** cho Export API: Thay skip bằng WHERE id > lastSeenId để latency flat tại mọi độ sâu trang.

9 Giao diện người dùng và quản trị hệ thống

Giao diện người dùng của trực tích hợp dữ liệu được thiết kế dưới dạng một trang quản trị tập trung, cung cấp cho quản trị viên khả năng giám sát, điều khiển và bảo vệ luồng dữ liệu của toàn bộ hệ thống. Giao diện được chia thành ba phân hệ chính, tương

ứng với các nghiệp vụ cốt lõi: bảng điều khiển trung tâm, quản lý lịch sử đồng bộ và quản lý sao lưu dữ liệu.

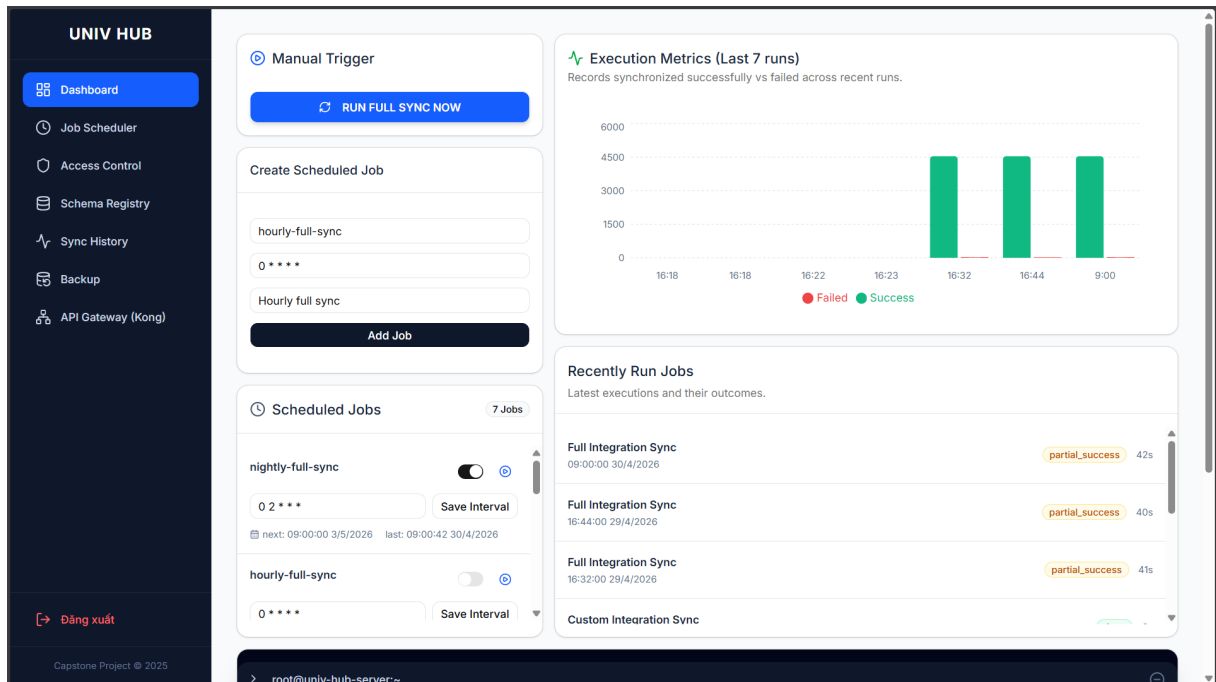
9.1 Bảng điều khiển trung tâm

Bảng điều khiển trung tâm là nơi cung cấp cái nhìn tổng quan nhất về trạng thái hoạt động của trực tích hợp theo thời gian thực. Phân hệ này được thiết kế với bốn thành phần chức năng chính:

1. **Kích hoạt đồng bộ thủ công:** Quản trị viên có thể kích hoạt tiến trình đồng bộ ngay lập tức. Hệ thống hỗ trợ hai chế độ: đồng bộ toàn phần hoặc đồng bộ tùy chỉnh (chỉ chọn một số bảng dữ liệu cụ thể thông qua hộp thoại lựa chọn). Trong quá trình xử lý, nút kích hoạt sẽ bị vô hiệu hóa tạm thời để tránh việc gọi hàm lặp lại.
2. **Quản lý tác vụ định kỳ:** Hệ thống cung cấp giao diện để tạo mới, bật/tắt và cập nhật các tác vụ tự động dựa trên biểu thức Cron. Với mỗi tác vụ, hệ thống hiển thị rõ ràng thông tin về thời gian thực thi gần nhất và thời gian dự kiến cho lần chạy tiếp theo.
3. **Thống kê chỉ số thực thi:** Sử dụng biểu đồ cột để trực quan hóa số lượng bản ghi đồng bộ thành công và thất bại trong bảy lần chạy gần nhất. Chức năng này giúp quản trị viên dễ dàng nhận biết xu hướng và phát hiện các dấu hiệu bất thường của hệ thống.
4. **Cửa sổ giám sát thời gian thực:** Đây là một điểm nhấn kỹ thuật của giao diện. Hệ thống thiết lập một kết nối liên tục đến máy chủ thông qua công nghệ Server-Sent Events (SSE). Nhờ đó, mọi nhật ký hoạt động từ máy chủ đều được đẩy trực tiếp lên giao diện và tự động cuộn xuống dòng mới nhất, giúp quản trị viên theo dõi sát sao từng bước của tiến trình ETL mà không cần làm mới trang.

9.2 Quản lý lịch sử đồng bộ

Do tính chất phức tạp của việc tích hợp nhiều nguồn dữ liệu khác nhau, phân hệ quản lý lịch sử đồng bộ được thiết kế theo mô hình phân cấp chính - phụ nhằm tối ưu hóa không gian hiển thị:



Hình 9.1: Giao diện bảng điều khiển trung tâm

- **Mức tiến trình:** Hiển thị danh sách các lần thực thi, bao gồm thông tin về tên tiến trình, thời gian bắt đầu, tổng thời lượng thực thi, trạng thái tổng thể (thành công, đang chạy, lỗi một phần, thất bại) và tổng số lượng bản ghi được xử lý.
- **Mức bảng dữ liệu:** Khi mở rộng một dòng tiến trình, giao diện sẽ hiển thị chi tiết kết quả đồng bộ của từng bảng dữ liệu riêng biệt. Quản trị viên có thể biết chính xác bảng nào được cập nhật bao nhiêu bản ghi, bảng nào bị bỏ qua hoặc bảng nào gặp lỗi.
- **Truy xuất nhật ký lỗi nguyên bản:** Đối với các bản ghi hoặc bảng dữ liệu gặp sự cố, quản trị viên có thể mở hộp thoại xem chi tiết lỗi hệ thống. Hệ thống trích xuất và hiển thị thông báo lỗi gốc từ bộ phân tích hoặc cơ sở dữ liệu, hỗ trợ tối đa cho quá trình gỡ lỗi.

9.3 Quản lý sao lưu và phục hồi

An toàn dữ liệu là yếu tố kiên quyết trong trực tích hợp. Phân hệ quản lý sao lưu cung cấp các công cụ tương tác trực tiếp với hệ thống lưu trữ tập tin. Giao diện phân loại các tệp sao lưu thành bốn nhóm tương ứng với ngữ cảnh tạo ra chúng:

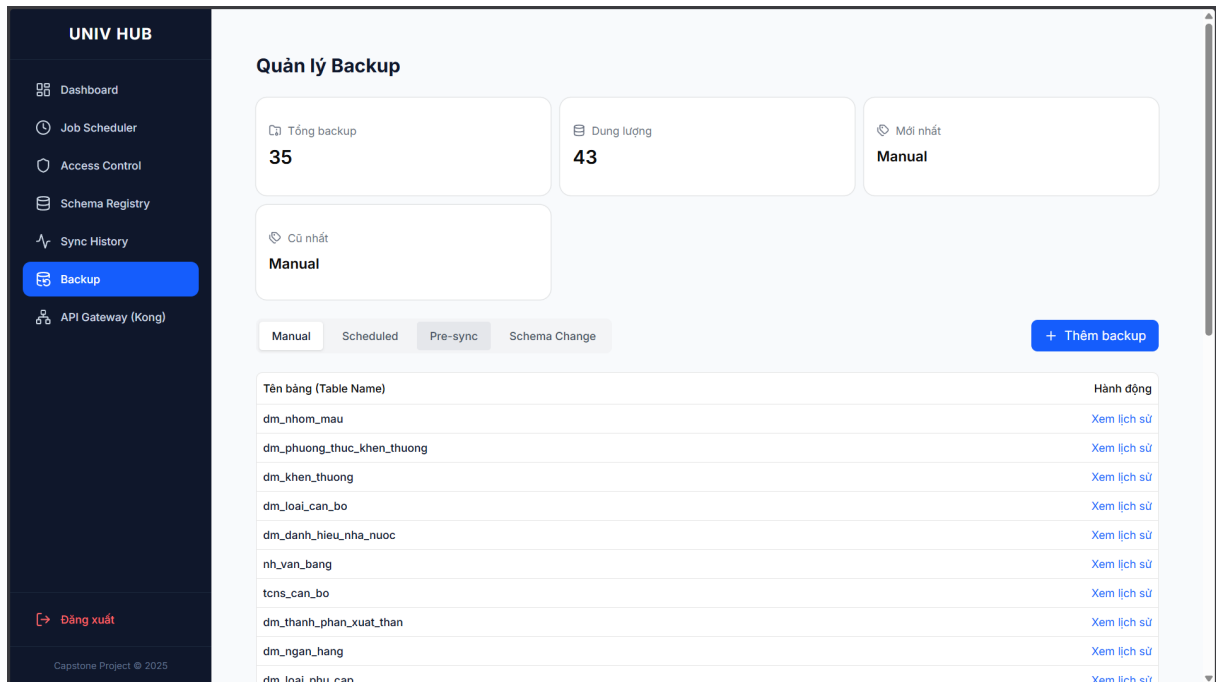
UNIV HUB		Lịch sử Đồng bộ (Sync Logs)				
- Dashboard - Job Scheduler - Access Control - Schema Registry Sync History - Backup - API Gateway (Kong)		Quản lý lịch sử các tiến trình ETL và chi tiết từng bảng dữ liệu. Làm mới dữ liệu				
Danh sách Tiến trình Cron Job						
Tên Tiến trình (Job)	Bắt đầu lúc	Thời lượng	Trạng thái	Tổng Record	Hành động	
Full Integration Sync	09:00:00 30/04/2026	42.29s	Lỗi một phần	4541 OK 25 Lỗi	Log Tho	
Chi tiết đồng bộ theo Bảng (Tables)						
Tên Bảng	Trạng thái	Record đã lưu	Ghi chú (Lỗi)			
dm_gioi_tinh	Thất bại	-	[SCHEMA CONTRACT VIOLATION] Payload có field chưa định nghĩa trong me			
dm_chuc_danh_khoa_hoc	Thất bại	-	No real data endpoint configured			
dm_dt_chung_chi ngoai ngu	Thất bại	-	No real data endpoint configured			
dm_dt_doi tuong_anqp	Thất bại	-	No real data endpoint configured			
dm_hinh_thuc_ky_luat	Thành công	+7				
dm_nhom_luong	Thất bại	-	No real data endpoint configured			
dm_nhom_mau	Thành công	+5				
dm_quan_he_gia_dinh	Thành công	+21				
dm_trinh_do_pho_thong	Thất bại	-	No real data endpoint configured			
dm_vi_tri_viec_lam	Thất bại	-	No real data endpoint configured			
tcns_ly_lich	Thất bại	-	No dataFromApi defined.			
nguoi_hoc	Thành công	+0				
dm_doi_tuong_chinh_sach	Thất bại	-	No real data endpoint configured			

Hình 9.2: Giao diện quản lý lịch sử đồng bộ

- Sao lưu thủ công:** Các bản sao lưu do quản trị viên chủ động tạo ra thông qua giao diện.
- Sao lưu định kỳ:** Các bản sao lưu được tạo bởi hệ thống theo lịch trình tự động.
- Trước đồng bộ:** Các bản chụp trạng thái dữ liệu được hệ thống tự động tạo ra ngay trước khi tiến hành ghi đè dữ liệu mới từ nguồn.
- Lược đồ thay đổi:** Các bản sao lưu được tạo ra khi hệ thống phát hiện có sự thay đổi về cấu trúc bảng từ nguồn.

Quy trình quản lý sao lưu được thiết kế theo luồng hai bước để tránh làm rối giao diện. Ở bước đầu tiên, hệ thống liệt kê danh sách các bảng dữ liệu hiện có bản sao lưu. Khi quản trị viên chọn một bảng cụ thể, hệ thống mới hiển thị danh sách chi tiết các tệp sao lưu của bảng đó theo mốc thời gian.

Tại đây, quản trị viên có thể tải tệp sao lưu trực tiếp về máy tính cá nhân thông qua cơ chế URL định tuyến an toàn hoặc nhấn nút phục hồi dữ liệu. Để đảm bảo an toàn, mọi thao tác phục hồi đều yêu cầu một bước xác nhận cảnh báo từ người dùng nhằm tránh việc ghi đè dữ liệu hiện tại do sơ suất.



Hình 9.3: Giao diện quản lý sao lưu và phục hồi dữ liệu

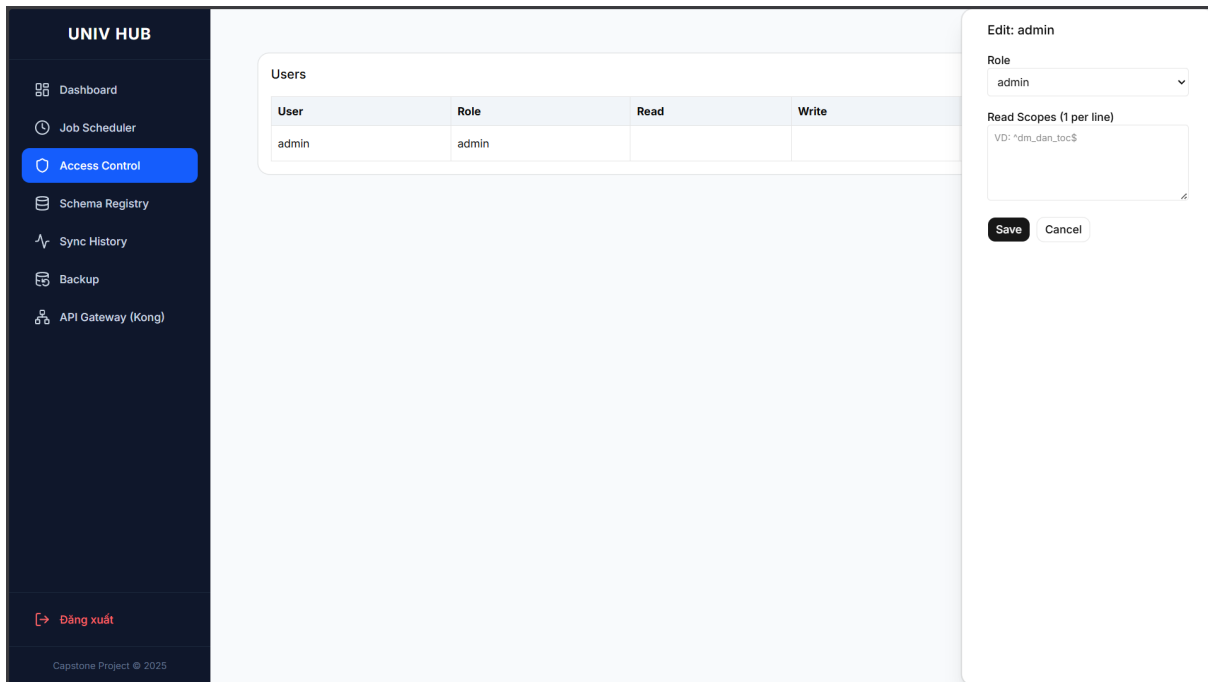
9.4 Quản lý phân quyền truy cập

Để đảm bảo tính bảo mật và ngăn chặn việc lạm dụng quyền hạn trong thao tác với dữ liệu, trực tích hợp cung cấp một phân hệ quản lý phân quyền dựa trên vai trò người dùng. Thông qua giao diện này, quản trị viên cấp cao có thể dễ dàng kiểm soát và giới hạn phạm vi truy cập đối với từng cá nhân hoặc hệ thống bên thứ ba khi họ tương tác với dữ liệu trên trực tích hợp.

Giao diện quản lý phân quyền bao gồm một bảng tổng hợp danh sách người dùng, hiển thị trực quan vai trò hiện tại (như quản trị viên, người dùng đọc, người dùng ghi) và các quyền hạn cụ thể đã được cấp phát.

Khi cần thay đổi quyền hạn, quản trị viên có thể mở bảng điều khiển chỉnh sửa bên cạnh (side-panel). Tại đây, ngoài việc thay đổi vai trò tổng thể, hệ thống còn cho phép thiết lập các quyền truy cập chi tiết đến từng bảng dữ liệu (Scope) thông qua cơ chế biểu thức chính quy (Regular Expression). Cụ thể:

- **Phạm vi đọc:** Danh sách các bảng dữ liệu mà tài khoản được phép truy vấn. (Ví dụ: thiết lập `dm_.*` cho phép người dùng đọc tất cả các bảng danh mục).
- **Phạm vi ghi:** Danh sách các bảng dữ liệu mà tài khoản được phép chèn, cập nhật hoặc xóa (chỉ áp dụng đối với những người dùng được cấp vai trò cho phép thao tác ghi).



Hình 9.4: Giao diện quản lý phân quyền truy cập cho người dùng

Cơ chế phân quyền này giúp phân tách rõ ràng ranh giới dữ liệu giữa các phòng ban, đảm bảo phòng ban nào chỉ được xem và xử lý đúng phần dữ liệu thuộc thẩm quyền của mình.

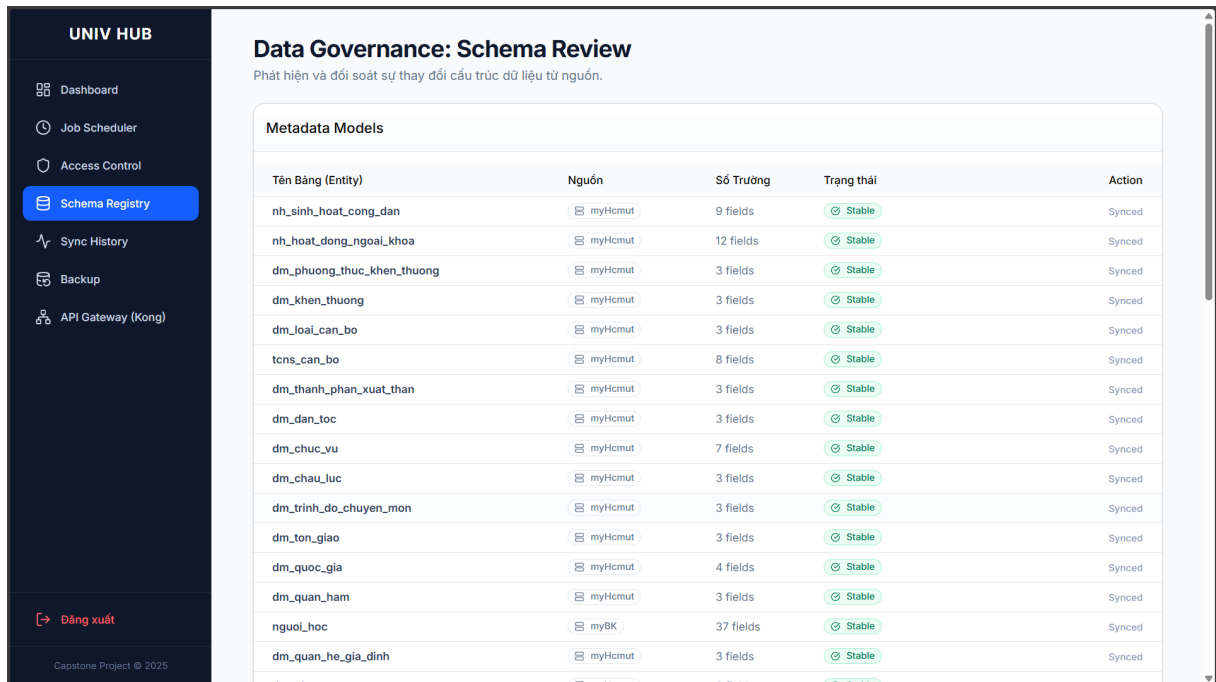
9.5 Quản lý và đối soát lược đồ dữ liệu

Trong kiến trúc tích hợp dữ liệu phân tán, việc các hệ thống nguồn tự ý thay đổi cấu trúc bảng (thêm, sửa, xóa cột) mà không thông báo trước là một trong những nguyên nhân hàng đầu gây sụp đổ luồng dữ liệu. Để giải quyết rủi ro này, hệ thống cung cấp một phân hệ quản lý và đối soát lược đồ dữ liệu chuyên sâu.

Giao diện hiển thị danh sách tất cả các bảng dữ liệu mà trực tích hợp đang theo dõi, kèm theo trạng thái ổn định của từng bảng. Nếu hệ thống nguồn có sự thay đổi về cấu trúc trả về, trạng thái của bảng sẽ lập tức chuyển sang cảnh báo thay đổi (*Schema Drift*), đồng thời mọi tiến trình đồng bộ dữ liệu vào bảng này sẽ tự động bị đình chỉ tạm thời để bảo vệ cơ sở dữ liệu đích.

Hộp thoại so sánh lược đồ dữ liệu được lấy cảm hứng từ GitHub. Khi quản trị viên chọn xem xét một bảng đang bị cảnh báo, hệ thống sẽ thực hiện các thuật toán tính toán và hiển thị trực quan sự khác biệt:

- **So sánh song song:** Giao diện chia thành hai cột, một bên là cấu trúc mới vừa



Tên Bảng (Entity)	Nguồn	Số Trường	Trạng thái	Action
nh_sinh_hoat_cong_dan	myHcmut	9 fields	Stable	Synced
nh_hoat_dong_ngoai_khoa	myHcmut	12 fields	Stable	Synced
dm_phuong_thuc_khen_thuong	myHcmut	3 fields	Stable	Synced
dm_khen_thuong	myHcmut	3 fields	Stable	Synced
dm_loai_can_bo	myHcmut	3 fields	Stable	Synced
tcns_can_bo	myHcmut	8 fields	Stable	Synced
dm_thanh_phan_xuat_than	myHcmut	3 fields	Stable	Synced
dm_dan_toc	myHcmut	3 fields	Stable	Synced
dm_chuc_vu	myHcmut	7 fields	Stable	Synced
dm_chau_luc	myHcmut	3 fields	Stable	Synced
dm_trinh_do_chuyen_mon	myHcmut	3 fields	Stable	Synced
dm_ton_giao	myHcmut	3 fields	Stable	Synced
dm_quoc_gia	myHcmut	4 fields	Stable	Synced
dm_quan_ham	myHcmut	3 fields	Stable	Synced
nguai_hoc	myBK	37 fields	Stable	Synced
dm_quan_he_gia_dinh	myHcmut	3 fields	Stable	Synced

Hình 9.5: Giao diện quản lý lược đồ dữ liệu

nhận được từ nguồn, bên còn lại là cấu trúc cũ đang được lưu trữ trong cơ sở dữ liệu.

- **Phân loại thay đổi:** Các trường dữ liệu được đánh dấu màu sắc rõ ràng tương ứng với ba trạng thái: mới được thêm vào, đã bị thay đổi (như đổi kiểu dữ liệu hoặc độ dài) và đã bị xóa bỏ.
- **Cảnh báo rủi ro cao:** Nếu hệ thống phát hiện có trường dữ liệu bị xóa hoặc thay đổi kiểu (có nguy cơ gây mất dữ liệu hiện có), một thông báo cảnh báo màu đỏ sẽ xuất hiện, yêu cầu quản trị viên hết sức cẩn trọng.

Hơn thế nữa, để tự động hóa tối đa quy trình cập nhật, giao diện còn tích hợp tính năng tự động sinh mã nguồn (Auto-generate Code). Hệ thống sẽ chuyển đổi các kiểu dữ liệu từ cấu trúc của hệ thống nguồn sang định dạng chuẩn của Prisma ORM và tạo sẵn đoạn mã khai báo mô hình dữ liệu (Prisma Model). Quản trị viên chỉ cần nhấn nút sao chép, dán vào mã nguồn của trực tích hợp và chạy lệnh đồng bộ cơ sở dữ liệu mà không cần phải tự viết tay lại từ đầu.

Sau khi quá trình cập nhật cấu trúc cơ sở dữ liệu hoàn tất một cách an toàn, quản trị viên sẽ nhấn nút xác nhận trên giao diện. Lúc này, trạng thái của bảng sẽ trở lại trạng thái ổn định và các luồng đồng bộ dữ liệu sẽ tự động hoạt động trở lại bình thường.



Tài liệu

- [1] Andrea Cali, Diego Calvanese, Giuseppe De Giacomo, and Maurizio Lenzerini. Data integration under integrity constraints. *Information Systems*, 2004.
- [2] Weiguo Fan, Hongjun Lu, Stuart E. Madnick, and David Cheung. Discovering and reconciling value conflicts for numerical data integration. *Information Systems*, 2001.
- [3] Walaa S. Ismail, Mona M. Nasr, Torky I. Sultan, and Ayman E. Khedr. Semantic conflicts reconciliation as a viable solution for semantic heterogeneity problems.
- [4] Ee-Peng Lim and Roger H. L. Chiang. The integration of relationship instances from heterogeneous databases.